

# Amortized Deduction for Binary Linear Systems: Learning Branching Guidance from Exact Conditional Marginals

neuroMCTS project<sup>a</sup>

<sup>a</sup>*Korea University, Seoul, Republic of Korea*

---

## Abstract

A binary linear system (BLS) asks for  $\mathbf{x} \in \{0, 1\}^n$  satisfying  $A\mathbf{x} = \mathbf{b}$  with  $A \in \{0, 1\}^{m \times n}$ , a formulation that arises in laser-based non-destructive testing, where a binary occupancy image is reconstructed from integer projections. On market-split-style instances the linear relaxation is deliberately uninformative, so backtracking search spends most of its effort on branching decisions that a cheap heuristic cannot make well. We present a learned branching heuristic for this setting and an exact procedure for training it.

The method rests on two observations. First, conditioning a BLS on a partial assignment is *identical* to substituting the assignment out and obtaining a smaller BLS; a single network therefore predicts at the root and at every interior node without architectural change, and training states are ordinary instances of reduced size. Second, on instances small enough to enumerate, the exact conditional marginal  $p_j = \Pr[x_j = 1 \mid A, \mathbf{b}]$  is computable, and it partitions the free variables of a node into those every surviving solution agrees on — where branching the other way empties the subtree and forces a backtrack — and those solutions disagree on, where either branch still reaches a solution. Only the first kind carries cost. This partition supplies both an exact training target and a criterion for which predictions matter: at the root only 6.3% of variables are of the first kind, but by depth 8 the share is 87.9%, which is where prediction accuracy begins converting into search cost.

We train a 21.8K-parameter bipartite graph network on these targets and use it for branching only, never for fixing variables, since it produces a probability rather than a proof. Against LP-probing — the sound procedure that detects the same variables, at one linear program each — the network needs

a single relaxation and one forward pass, so their cost ratio grows linearly in instance size:  $1.7\times$  at  $n = 25$ ,  $7.2\times$  at  $n = 60$ ,  $17.8\times$  at  $n = 150$ , measured with a warm-started backend on both sides. Under a matched wall-clock budget on a warm-started search loop, replicated over three seeds, the resulting search is  $2.6\text{--}3.0\times$  faster in median than LP-guided branching at  $21 \times 60$  (faster on 23/30 instances in every seed, sign test  $p = 0.005$ ) and expands  $9.5\times$  fewer nodes, with both strategies solving every instance; within the full pipeline, where lattice reduction runs first, it closes the residual  $3.0\times$  faster than LP-guided branching, while random branching closes none of it. Controlling the number of solutions across sizes, we find no measurable transfer penalty: a network trained only at  $10 \times 25$  is statistically indistinguishable from one trained at the evaluation size. On a 100-instance  $21 \times 60$  set the supervised guidance is  $2.0\text{--}2.3\times$  faster than LP-guided branching with  $5.5\times$  fewer nodes. Two further experiments locate what the supervised target does and does not capture. Training on states of deployment size — the regime where the network’s prediction margin lives — does *not* help: it loses competence on the small interior states where most nodes are spent, and the search gets slower. What does help is optimizing the objective the search actually pays: with the exact subtree cost of every branching decision available from the search itself, an actor-critic fine-tuning of the branching policy, initialized at the supervised rule, expands  $2.3\times$  fewer nodes and is  $2.0\text{--}2.2\times$  faster than the supervised policy on the same 100 instances ( $p < 0.01$ ),  $4.8\text{--}5.9\times$  faster than LP-guided branching. The fine-tuned policy branches on a different variable than the confidence rule at 95% of held-out states, typically one it is *less* sure about, which is direct evidence that the probability of being right and the cost of being wrong are different objectives.

We also report what the method does not achieve. CP-SAT solves the same instances  $9\text{--}40\times$  faster at every size tested ( $9\text{--}10\times$  at  $21 \times 60$  after fine-tuning), and our system does not compete with it; decomposing that gap shows our branching produces  $2.2\times$  *fewer* nodes but processes them  $32\times$  more slowly, which places the deficit in per-node cost rather than in branching quality, though we do not show that it is recoverable. And the cost advantage over probing, while asymptotically real, is only  $1.3\text{--}1.7\times$  at the depths where prediction matters most on our instances, where probing additionally offers proofs rather than predictions. The contribution we claim is therefore methodological: exact conditional posteriors are an obtainable and well-posed supervision signal for branching, in-tree states are a better training distribution than root states — a root-trained network is worse than

relaxation rounding when deployed in the tree, an in-tree-trained one is better by 3.4 points — and the resulting heuristic transfers across instance size. Code, generators, exact-marginal labels and per-instance results are released.

*Keywords:* Binary linear systems, Learning to branch, Amortized optimization, Exact conditional marginals, Market split, Graph neural networks

---

## 1. Introduction

### 1.1. Problem

Let  $A \in \{0, 1\}^{m \times n}$  and  $\mathbf{b} \in \mathbb{Z}^m$ . A *binary linear system* (BLS) is the feasibility problem

$$\text{find } \mathbf{x} \in \{0, 1\}^n \text{ such that } A\mathbf{x} = \mathbf{b}, \quad (1)$$

whose solution set we write  $\mathcal{S} = \{\mathbf{x} \in \{0, 1\}^n : A\mathbf{x} = \mathbf{b}\}$ . Each row is a cardinality constraint  $\sum_j a_{ij}x_j = b_i$  stating how many cells of a designated subset are occupied.

The motivating application is laser-based non-destructive testing. A specimen is scanned along  $m$  directions; each scan returns the integer count of absorbing cells along its path, and the task is to reconstruct the binary occupancy image. Two properties of that setting shape the problem formulation. Solutions need not be unique — several images can be consistent with the same measurements, and recovering *any* member of  $\mathcal{S}$  solves the physical problem. And measurement noise can perturb  $\mathbf{b}$  so that  $\mathcal{S} = \emptyset$ , making a reliable infeasibility verdict as valuable as a solution.

Deciding  $\mathcal{S} \neq \emptyset$  is NP-complete in general. The instances studied here are drawn from a deliberately hard regime modelled on market-split problems [6, 2], in which the linear relaxation

$$\tilde{\mathcal{S}} = \{\mathbf{x} \in [0, 1]^n : A\mathbf{x} = \mathbf{b}\} \quad (2)$$

is a large polytope with predominantly fractional vertices. Relaxation-based bounding is therefore weak, and a complete solver must decide where to branch with little guidance from the relaxation — exactly the decision this paper learns.

### 1.2. Where a learned component attaches, and what to train it on

A complete solver for (1) alternates deduction (constraint propagation, relaxation-based pruning) with a *branching decision*: which free variable to split on, and which value to try first. Deduction is sound and must remain so; branching is a heuristic choice, where an error costs search time but never correctness. This asymmetry makes branching the natural place to attach a learned component, and it is where learning has been most successful in mixed-integer programming [9, 7].

The difficulty is supervision. Two standard answers are available and both are unsatisfactory here. Training against a planted solution  $\mathbf{x}^*$  treats one member of  $\mathcal{S}$  as ground truth, which is ill-posed when  $|\mathcal{S}| > 1$ : nothing in  $(A, \mathbf{b})$  distinguishes the planted solution from any other, so the label is a sample from the posterior rather than the posterior. Imitating an expensive expert, as in strong-branching imitation, requires an expert whose decisions are both informative and affordable to collect; the natural candidate here, LP-probing, costs one linear program per variable per node.

We take a third route, available because these instances have planted solutions and because feasibility can be enumerated at moderate size: compute the *exact conditional posterior* by exhaustive enumeration on small instances, and train against it. Section 3 shows that this is both tractable and, in a precise sense, the right target — and that the posterior additionally identifies *which* predictions can affect search at all.

### 1.3. Contributions

1. **A reduction identity that makes in-tree supervision trivial to implement.** Conditioning (1) on a partial assignment yields a smaller BLS with the same structure (Proposition 1). One architecture, one label generator and one training loop therefore serve the root and every interior node, and a depth- $d$  training state is stored as an ordinary instance on  $n - d$  variables.
2. **A partition of branching decisions by whether they can cost anything.** The exact marginals split a node’s free variables into DET (all surviving solutions agree; branching the other way empties the subtree) and OPEN (solutions disagree; either branch reaches a solution). Only DET errors cost a backtrack. We measure the composition: 6.3% DET at the root, rising to 87.9% at depth 8 (Sections 4.3, 4.4). This locates useful prediction inside the tree rather than at the root and explains why root-level accuracy is a poor objective.

- 60    3. **A cost characterization of the learned heuristic against the**  
 61    **sound procedure it approximates.** The network needs one relax-  
 62    ation and one forward pass; LP-probing needs one relaxation per vari-  
 63    able. Measured with a warm-started backend on both sides, the ratio  
 64    scales from  $1.7\times$  at  $n = 25$  to  $17.8\times$  at  $n = 150$  (Section 4.5). We  
 65    report this as a scaling law rather than a single figure, and note that  
 66    at the sizes where our search operates the advantage is modest and  
 67    probing has the additional merit of soundness.
- 68    4. **End-to-end evaluation under matched wall-clock budgets.** At  
 69     $21 \times 60$  on a warm-started search loop, replicated over three seeds, the  
 70    learned guidance is  $2.0\text{--}2.3\times$  faster in median than LP-based branching  
 71    on a 100-instance set (faster on 70–71/100, sign test  $p < 0.001$ ) and  
 72    expands  $5.5\times$  fewer nodes; on the original 30-instance set the margins  
 73    are  $2.6\text{--}3.0\times$  and  $9.5\times$ , and both strategies solve every instance within  
 74    600 s (Section 4.6). In the deployed cascade both sound strategies close  
 75    every residual instance, the learned one  $3.0\times$  faster (40/50 wins pooled  
 76    over seeds,  $p = 2 \times 10^{-5}$ ), while random branching closes none of the  
 77    50 residual instances (Section 4.12).
- 78    5. **Size transfer with solution multiplicity controlled.** Holding  $|\mathcal{S}|$   
 79    comparable across sizes, a network trained only at  $10 \times 25$  matches one  
 80    trained at the evaluation size  $21 \times 60$  (faster on 15/30 instances; total  
 81    time differing by 0.4%), showing the learned quantity is not size-specific  
 82    (Section 4.8).
- 83    6. **A negative result on aligning the training distribution with**  
 84    **deployment.** The network’s margin over relaxation rounding lives on  
 85    states with 50–60 free variables, outside its 13–25 training range. Train-  
 86    ing on exactly those states, with everything else fixed, raises accuracy  
 87    there by about one point and *doubles* the search tree, because it loses  
 88    competence on the small interior states where two thirds of all nodes  
 89    are spent (Section 4.10).
- 90    7. **Cost-aware fine-tuning of the branching policy.** The exact cost  
 91    of a branching decision — the number of nodes its subtree expands —  
 92    is available from the search itself, and on held-out states the confidence  
 93    rule picks the cheapest of its five candidates only 22% of the time.  
 94    An actor-critic fine-tuning on that cost, initialized at the supervised  
 95    rule with the marginal network frozen, expands  $2.3\times$  fewer nodes and  
 96    is  $2.0\text{--}2.2\times$  faster than the supervised policy on the 100-instance set  
 97    (70/100 and 64–67/100 wins,  $p < 0.01$  in every seed),  $4.8\text{--}5.9\times$  faster

98 than LP-guided branching (Section 4.11).

## 99 2. Related Work

### 100 2.1. Binary linear systems and market-split hardness

101 Problem (1) is a 0/1 integer feasibility problem and is NP-complete for  
 102 general  $A$ . Practical hardness, however, depends on the instance distribution.  
 103 Cornuéjols and Dawande [6] introduced the market-split family, in which  $m$   
 104 agents must divide  $n$  items so that every agent’s budget is met exactly; with  
 105  $n \approx 10(m - 1)$  these instances defeat branch-and-bound at remarkably small  
 106 sizes. The mechanism is the integrality gap: the relaxation polytope is large  
 107 and its vertices are fractional, so bounding prunes little and the search must  
 108 branch nearly blindly. Aardal et al. [2] showed that lattice basis reduction  
 109 attacks the same family far more effectively than branch-and-bound, a result  
 110 we rely on in Section 3.2.

111 Our generator (Section 4.1) makes this mechanism explicit by maximizing  
 112 the spread of relaxation vertices, which enlarges  $\tilde{\mathcal{S}}$  and weakens the relaxation  
 113 signal by construction.

### 114 2.2. Complete solvers

115 *Mixed-integer programming.* Branch-and-bound over the linear relaxation,  
 116 strengthened by presolve and cutting planes, is the standard approach. Its  
 117 leverage comes from pruning nodes whose relaxation is infeasible or bounded  
 118 away from the target, which is precisely what the market-split construction  
 119 removes.

120 *Constraint programming with clause learning.* CP-SAT-style solvers com-  
 121 bine constraint propagation with conflict-driven clause learning [12, 14] and  
 122 lazy clause generation [15]. Their strength on this family comes from a  
 123 property central to our argument: a search *manufactures* information as it  
 124 descends. A marginal  $\Pr[x_j = 1 \mid A, \mathbf{b}]$  may be uninformative at the root  
 125 while  $\Pr[x_j = 1 \mid A, \mathbf{b}, x_1=1, x_5=0]$  is 0 or 1. Structure invisible at the root  
 126 becomes deducible after commitments are made, which is why Section 3.4  
 127 studies the interior of the tree rather than the root.

128 *Lattice reduction.* An alternative attack embeds (1) in a lattice and searches  
 129 for short vectors. LLL [10] reduces a basis in polynomial time and BKZ  
 130 [16] generalizes it with a block size trading time against basis quality. The  
 131 Aardal–Hurkens–Lenstra embedding [1, 2] is constructed so that a 0/1 so-  
 132 lution appears directly as a short vector, making reduction a certificate-  
 133 producing procedure rather than a heuristic. We use it as the first stage  
 134 of the deployed pipeline. Lattice enumeration remains the state of the art  
 135 on the canonical benchmark: Wassermann [35] solves QOBLIB market-split  
 136 instances up to  $m = 14$  on one CPU where branch-and-cut stops at  $m = 7$ ;  
 137 our instances are a different regime (multiplicity matched,  $m/n \approx 0.35$ ) and  
 138 our contribution is a branching methodology rather than a record there.

### 139 2.3. Learning for combinatorial optimization

140 Bengio et al. [4] survey this field and frame it in terms we adopt: solvers  
 141 rely on hand-crafted heuristics for decisions that are “otherwise too expensive  
 142 to compute or mathematically not well defined,” and learning is a candidate  
 143 for making those decisions better. The phrase *too expensive to compute*  
 144 presumes the decision is computable in principle and asks only about cost,  
 145 which is the situation we measure in Section 4.5.

146 *Marginals of the solution set as a branching signal.* The fraction of solutions  
 147 agreeing with a decision was a branching criterion before learning: counting-  
 148 based search [32] branches on the variable–value pair of maximum solu-  
 149 tion density computed inside individual constraints, and belief-propagation-  
 150 guided decimation [29] fixes the most biased variable under approximate  
 151 marginals recomputed after each fixing. Learned versions replace the count-  
 152 ing by a network: NSNet [28] trains a BP-structured graph network on exact  
 153 marginals of a SAT formula’s satisfying assignments, obtained by enumer-  
 154 ation, and rounds them into a local-search initialization; MIP-GNN [27],  
 155 Neural Diving [30] and predict-and-search [26] predict per-variable biases  
 156 from collected near-optimal solutions and use them to fix variables, warm-  
 157 start or restrict a search. Our target is the same as NSNet’s; what differs is  
 158 that the marginals are conditional, carried to every node by the reduction  
 159 identity, and used to guide a complete search without fixing anything, and  
 160 that the DET/OPEN partition and the in-tree-versus-root comparison have no  
 161 counterpart there.

162 *Branching imitation.* The clearest success in this area is imitation of an  
 163 expensive expert. Strong branching scores candidates by tentatively solving

164 both children’s relaxations; it yields small trees and is too slow to run at  
 165 every node. Gasse et al. [7] train a bipartite graph network to imitate it and  
 166 recover most of the benefit at a fraction of the cost, following Khalil et al.  
 167 [9]. The gain is not new information — the expert already knew the answer  
 168 — but cheap access to it. Our method occupies the same structural position,  
 169 with two differences: the target is an exact posterior rather than an expert’s  
 170 scores, and the analysis identifies which of the expert’s decisions matter.

171 *Direct satisfiability prediction..* Attempts to have a network decide satisfia-  
 172 bility end-to-end have a weaker record. NeuroSAT [17] classifies satisfiability  
 173 by message passing over a literal–clause graph and succeeds on small ran-  
 174 dom instances, but Li et al. [11] report substantial accuracy collapse under  
 175 size transfer. Chen et al. [5] prove that standard message-passing networks  
 176 cannot separate certain feasible/infeasible MILP pairs at all, since the pairs  
 177 are indistinguishable in the Weisfeiler–Lehman hierarchy that bounds such  
 178 architectures [36]. We therefore do not ask the network for a feasibility ver-  
 179 dict; it supplies branching guidance inside a complete search that remains  
 180 responsible for soundness.

181 *Graph representations..* We use graph attention [18] over the bipartite variable–  
 182 constraint graph, the standard encoding for MILP in this literature [7].

183 *Reinforcement learning for branching..* Imitation targets an expert’s rank-  
 184 ing, not the search cost the ranking is meant to reduce. FMSTS [20] uses  
 185 a node’s subtree size as an observable Q-value, shows that under depth-  
 186 first search minimizing every subtree minimizes the whole tree, and learns  
 187 from scratch; tree MDPs [21] formalize that per-subtree credit assignment;  
 188 retro branching [31] learns from retrospectively extracted subtree trajecto-  
 189 ries; SORREL [25] initializes from suboptimal demonstrations; Neuro# [34]  
 190 trains a graph network on the residual formula of a #SAT solver by evolution  
 191 strategies to minimize branching decisions. In constraint programming, value  
 192 ordering has been learned from solutions [24] and by deep Q-learning [23],  
 193 and impact-based search [33] prefers the variable whose assignment shrinks  
 194 the search space most. Our fine-tuning stage (Section 3.8) uses the same  
 195 per-subtree credit with a plain policy-gradient update [19], and differs in its  
 196 starting point: the policy is initialized at a supervised rule derived from exact  
 197 conditional marginals, and the marginal network is frozen, so the only thing  
 198 the cost signal can change is which variable is branched on. The classical

statement of the objective is the fail-first principle [22]: choose the variable whose wrong branch is refuted soonest.

#### 2.4. Amortized optimization

The distinction organizing our results predates its use in combinatorial optimization. Gershman and Goodman [8] introduced *amortized inference*: when many related queries must be answered, solving each from scratch is wasteful and the question becomes how to reuse computation across queries. Amos [3] develops the same idea as *amortized optimization* — using learning to predict solutions by exploiting shared structure across similar instances — and surveys its appearance in variational inference, meta-learning and control. Millidge [13] states the trade-off in the form most relevant here: *direct* optimization spends computation on the instance at hand and pays that cost at every invocation, whereas *amortized* optimization converts the problem into supervised learning with cheap inference but requires a corpus of solved instances up front and is bounded by the approximator’s generalization; the productive arrangement is hybrid, with amortized predictions steering a direct optimizer.

Our system instantiates that hybrid precisely. The direct method is LP-probing, sound and costly. The amortized method is the network, cheap and unsound. The hybrid rule is that the network steers branching while probing and propagation retain exclusive authority over variable fixing (Section 3.7).

### 3. Proposed Method

#### 3.1. Setting and notation

For a partial assignment fixing variables  $F \subseteq \{1, \dots, n\}$  to values  $\mathbf{v} \in \{0, 1\}^{|F|}$ , write

$$\mathcal{S}(F, \mathbf{v}) = \{\mathbf{x} \in \mathcal{S} : \mathbf{x}_F = \mathbf{v}\} \quad (3)$$

for the solutions consistent with it, and let  $K = \{1, \dots, n\} \setminus F$  index the free variables. The *conditional marginal* of a free variable  $j \in K$  is

$$p_j(F, \mathbf{v}) = \frac{|\{\mathbf{x} \in \mathcal{S}(F, \mathbf{v}) : x_j = 1\}|}{|\mathcal{S}(F, \mathbf{v})|}. \quad (4)$$

226 *3.2. System overview*

227 The deployed solver has two stages:

$$\text{AHL/BKZ lattice reduction} \rightarrow \text{guided backtracking search.} \quad (5)$$

228 The lattice stage attempts a certificate-producing reduction with a size-  
 229 dependent block parameter under up to ten random column permutations;  
 230 when it succeeds it returns a 0/1 vector that has been verified against  $A\mathbf{x} = \mathbf{b}$ ,  
 231 so it is sound. It costs 0.09–0.16 s per instance — 1% of end-to-end time at  
 232  $21 \times 60$  — and closes 30/30, 24–28/30 and 10–16/30 instances at  $10 \times 25$ ,  
 233  $18 \times 50$  and  $21 \times 60$ . Everything it leaves goes to the search.

234 Within a node of that search, three sound steps run before any learned  
 235 component is consulted: a bound check on the reduced right-hand side, unit  
 236 propagation, and the node relaxation’s infeasibility test (warm-started from  
 237 the parent basis). These are what fix variables and prune. The learned com-  
 238 ponent is consulted afterwards and only chooses which free variable to branch  
 239 on and which value to try first, so a wrong prediction costs backtracking and  
 240 nothing else. This places the

241 This placement matters for the evaluation: because the lattice stage is  
 242 independent of the branching heuristic, every heuristic under comparison  
 243 receives exactly the same residual set of instances, so conditioning on that  
 244 residual describes the algorithm rather than selecting favourable cases.

245 *3.3. Conditioning is reduction*

246 **Proposition 1.** *Let  $A_F$  and  $A_K$  denote the column submatrices of  $A$  indexed*  
 247 *by  $F$  and  $K$ . Then*

$$\mathcal{S}(F, \mathbf{v}) \cong \{\mathbf{y} \in \{0, 1\}^{|K|} : A_K \mathbf{y} = \mathbf{b} - A_F \mathbf{v}\}, \quad (6)$$

248 *the bijection being  $\mathbf{x} \mapsto \mathbf{x}_K$ . Consequently the conditional marginals (4)*  
 249 *of the original instance are the ordinary marginals of the reduced instance*  
 250  *$(A_K, \mathbf{b} - A_F \mathbf{v})$ .*

251 *Proof.*  $A\mathbf{x} = A_F \mathbf{x}_F + A_K \mathbf{x}_K$ . Fixing  $\mathbf{x}_F = \mathbf{v}$  gives  $A\mathbf{x} = \mathbf{b} \iff A_K \mathbf{x}_K =$   
 252  $\mathbf{b} - A_F \mathbf{v}$ , and  $\mathbf{x} \mapsto \mathbf{x}_K$  is a bijection between the two sets since  $\mathbf{x}_F$  is deter-  
 253 mined. The marginal statement follows by counting.  $\square$

254 The identity is elementary but it is what makes in-tree supervision practi-  
 255 cal. A depth- $d$  search node is not a special object requiring its own encoding;

256 it is an instance of the same problem on  $n - d$  variables. One network, one  
 257 feature extractor and one label generator therefore cover the whole tree, and  
 258 training data at depth  $d$  is produced by substitution rather than by instru-  
 259 menting a solver.

260 Two implementation details preserve correctness. Rows that become all-  
 261 zero after substitution carry no constraint and are removed; retaining them  
 262 causes linear-programming backends to reject the model. And the free bound  
 263 condition  $0 \leq \mathbf{b} - A_F \mathbf{v} \leq A_K \mathbf{1}$  is checked before any relaxation is solved,  
 264 since it detects a large fraction of dead nodes at negligible cost.

### 265 3.4. Which predictions can cost anything

266 At a node with surviving solution set  $\mathcal{S}(F, \mathbf{v}) \neq \emptyset$ , partition the free  
 267 variables by their conditional marginal:

$$\text{DET} = \{j \in K : p_j \in \{0, 1\}\}, \quad \text{OPEN} = \{j \in K : 0 < p_j < 1\}. \quad (7)$$

268 The distinction is operational rather than statistical:

- 269 • For  $j \in \text{OPEN}$ , both children of a branch on  $x_j$  contain solutions. Either  
 270 choice keeps a solution reachable, so *no branching decision on a OPEN*  
 271 *variable can cost anything*.
- 272 • For  $j \in \text{DET}$ , one child is empty. Branching against the forced value  
 273 guarantees that the subtree is exhausted without a solution, costing a  
 274 full backtrack.

275 This gives the conditional marginal its operational reading. It is not a  
 276 “probability of being correct” about some designated solution, but the *frac-*  
 277 *tion of the surviving solution set retained* by an assignment; and the only  
 278 errors that matter are those on variables where that fraction is 0 or 1.

279 Two consequences follow, both of which the experiments confirm. First,  
 280 an aggregate accuracy over all free variables is a poor objective, because it  
 281 mixes decisions that can cost a backtrack with decisions that cannot; the  
 282 relevant quantity is accuracy on DET. Second, since  $|\mathcal{S}(F, \mathbf{v})|$  shrinks as the  
 283 search descends, the DET share grows with depth. This alone does not say  
 284 *where* a learned predictor is useful, because a DET variable that the relaxation  
 285 already rounds correctly needs no predictor. The useful region is where DET  
 286 variables are plentiful *and* the relaxation is weak, and Section 4.9 shows that  
 287 this is governed by constraint density  $m/|K|$  rather than by depth as such:  
 288 on small instances the two coincide, on larger ones they do not.

### 289 3.5. Exact conditional marginals as training targets

290 For instances small enough to enumerate,  $\mathcal{S}$  is obtained exactly with a  
 291 constraint solver in all-solutions mode, and (4) is computed by counting.  
 292 Training minimizes the cross-entropy to the resulting Bernoulli targets,

$$\mathcal{L} = -\frac{1}{|K|} \sum_{j \in K} \left[ p_j \log \hat{p}_j + (1 - p_j) \log(1 - \hat{p}_j) \right], \quad (8)$$

293 which is minimized exactly at  $\hat{p}_j = p_j$ . Note that the same loss with a planted  
 294 solution in place of  $p_j$  — the conventional choice — is a noisy surrogate  
 295 whenever  $|\mathcal{S}| > 1$ , since the planted solution is one draw from the posterior;  
 296 on our instances a Bayes-optimal predictor agrees with it only about two  
 297 thirds of the time.

298 *Enumeration correctness.* A subtlety here is easy to get wrong and silently  
 299 corrupts every label derived from it. Constraint solvers report distinct sta-  
 300 tuses for “search exhausted” and “time limit reached with solutions already  
 301 collected.” Only the former certifies that  $\mathcal{S}$  is complete; accepting the latter  
 302 substitutes a truncated solution set, which biases every marginal computed  
 303 from it. In our implementation only **OPTIMAL** and **INFEASIBLE** are accepted  
 304 and timed-out instances are discarded rather than averaged in. A useful  
 305 diagnostic is that mean enumeration time equal to the time limit indicates  
 306 truncated instances being counted as complete.

307 *Sampling reachable states.* A depth- $d$  training state is built by drawing  $d$   
 308 uniformly from  $\{0, \dots, 12\}$ , taking a prefix of an LP-confidence variable or-  
 309 dering, and fixing those variables to the values of a randomly chosen *actual*  
 310 *solution*. Fixing arbitrary values would produce states with  $\mathcal{S}(F, \mathbf{v}) = \emptyset$ ; such  
 311 states are dead nodes that propagation prunes immediately and on which the  
 312 conditional marginal is undefined, so they are not states a branching heuris-  
 313 tic is ever asked about. The depth range is a design choice we did not tune;  
 314 on  $10 \times 25$  instances it yields training states with 13–25 free variables, and  
 315 Section 4.9 examines what happens when the deployed search queries states  
 316 outside that range, which at  $21 \times 60$  is most of them.

### 317 3.6. Network

318 An instance is encoded as a bipartite graph with  $n$  variable nodes,  $m$   
 319 constraint nodes, and an edge  $(j, i)$  whenever  $a_{ij} = 1$ . Let  $\tilde{\mathbf{x}}$  be the relaxation

320 solution of the current (reduced) instance and  $r_i = \sum_j a_{ij}$  the row sums.  
 321 Node features are

$$\text{variable } j : (\tilde{x}_j, \deg(j)/m, |\tilde{x}_j - \text{round}(\tilde{x}_j)|), \quad (9)$$

$$\text{constraint } i : (b_i/r_i, r_i/n, (b_i - A_i\tilde{\mathbf{x}})/r_i), \quad (10)$$

322 i.e. relaxation value, column degree and fractionality for variables; tight-  
 323 ness, scaled row size and relaxation residual for constraints. All are size-  
 324 normalized, which is what permits a single parameter set to apply at any  
 325  $(m, n)$ .

326 Both node types are embedded to  $h = 64$  dimensions by a linear map  
 327 and refined by  $L = 4$  rounds of bidirectional message passing with graph  
 328 attention [18]:

$$\mathbf{h}^{\text{con}} \leftarrow \text{LN}(\text{ELU}(\text{GAT}(\mathbf{h}^{\text{var}} \rightarrow \mathbf{h}^{\text{con}})) + \mathbf{h}^{\text{con}}), \quad (11)$$

$$\mathbf{h}^{\text{var}} \leftarrow \text{LN}(\text{ELU}(\text{GAT}(\mathbf{h}^{\text{con}} \rightarrow \mathbf{h}^{\text{var}})) + \mathbf{h}^{\text{var}}), \quad (12)$$

329 followed by a two-layer head emitting one logit per variable,  $\hat{p}_j = \sigma(z_j)$ . The  
 330 model has 21,761 parameters.

331 The architecture is deliberately small and single-headed. Its only out-  
 332 put is the quantity the objective (8) defines; there are no auxiliary value  
 333 or feasibility heads, because the search draws soundness from its symbolic  
 334 components and asks the network for nothing else.

### 335 3.7. Inference: guidance, not commitment

336 The network is consulted once per node. Branching selects

$$j^* = \arg \max_{j \in K} |\hat{p}_j - \tfrac{1}{2}|, \quad \text{first value} = \mathbb{K}[\hat{p}_{j^*} \geq \tfrac{1}{2}], \quad (13)$$

337 that is, the variable the network is most confident about, tried at its pre-  
 338 dicted value. The search remains complete and a wrong prediction costs  
 339 backtracking only.

340 Variables are *fixed* only by sound deduction. Two mechanisms are avail-  
 341 able: constraint propagation, and LP-probing, which fixes  $x_j$  when assign-  
 342 ing the opposite value makes even the relaxation infeasible. The division  
 343 of labour follows directly from the measurements in Section 4.5: where a  
 344 wrong answer costs time, the cheap unsound predictor is preferable; where  
 345 a wrong answer costs correctness, the expensive sound procedure is required  
 346 and cannot be replaced.

347 Algorithm 1 states the resulting search.

---

**Algorithm 1** Guided backtracking search for a BLS

---

**Input:** instance  $(A, \mathbf{b})$ , guidance model  $f_\theta$ , time budget  $T$

stack  $\leftarrow [(A, \mathbf{b})]$

**while** stack  $\neq \emptyset$  **and** elapsed  $< T$ :

$(A', \mathbf{b}') \leftarrow \text{pop}(\text{stack})$

**if**  $\mathbf{b}' < \mathbf{0}$  **or**  $\mathbf{b}' > A'\mathbf{1}$ : **continue** // *free bound check*

    drop all-zero rows of  $A'$ ; **if** none remain: **return** any completion

    propagate; **if** contradiction: **continue**

**if** propagation determined a set  $D$  of variables:

        push reduce( $A', \mathbf{b}', D$ ); **continue** // *Proposition 1*

**if** all variables assigned: **return** solution if verified, else **continue**

$\hat{\mathbf{p}} \leftarrow f_\theta(A', \mathbf{b}')$  // *one forward pass, all variables*

$j^* \leftarrow \arg \max_j |\hat{p}_j - 1/2|$ ;  $v \leftarrow \mathbb{K}[\hat{p}_{j^*} \geq 1/2]$

    push reduce( $A', \mathbf{b}', \{j^*=1-v\}$ ), then reduce( $A', \mathbf{b}', \{j^*=v\}$ )

**return** unresolved

---

348 *3.8. Cost-aware fine-tuning of the branching policy*

349 The rule of Algorithm 1 branches on the variable the network is surest  
350 about. That maximizes the probability that the first child contains a solution;  
351 it says nothing about how much the search pays when the first child does not,  
352 nor about how much smaller the children are. Both quantities are properties  
353 of the *subtree*, and a search produces them as a by-product: the cost of a  
354 branching decision at node  $u$  is the number of nodes expanded below  $u$  until  
355 a solution is found there or the subtree is exhausted, and depth-first search  
356 returns it exactly when it backtracks out of  $u$ . We use this cost to fine-tune  
357 which variable is branched on.

358 *Decision process..* A state is the reduced instance  $(A', \mathbf{b}')$  after propagation;  
359 an action is a free variable  $j$ ; the first value tried remains  $\mathbb{K}[\hat{p}_j \geq 1/2]$  from  
360 the marginal network, which is not modified. The return of the action at  $u$   
361 is  $-\log c(u)$  with  $c(u)$  the subtree cost. Because every decision has its own  
362 subtree, credit is assigned per decision rather than per episode, which is what  
363 made the sparse-reward formulations we tried earlier unlearnable.

364 *Policy and value..* On the frozen MarginalNet trunk we add two heads. The  
365 policy logit is

$$g_j(s) = \log|\hat{p}_j - \tfrac{1}{2}| + h_\phi(\mathbf{h}_j), \quad (14)$$

where  $h_\phi$  is a two-layer network on the variable embedding  $\mathbf{h}_j$  whose last layer is initialized to zero, so that  $\pi(j \mid s) \propto \exp(g_j/\tau)$  starts as a softening of the supervised rule and its argmax is exactly Algorithm 1. The value head  $V_\psi(s)$  reads a pooled graph embedding together with  $\log |K|$  and  $\log m$  and predicts  $\log c$ .

*Update..* Each epoch samples training states, rolls the sampled policy ( $\tau = 1$ ) out to a solution, and records every decision with its exact cost. With advantage  $\alpha(u) = V_\psi(s_u) - \log c(u)$ , standardized within the epoch, the update is

$$\mathcal{L} = \sum_u \left[ -\alpha(u) \log \pi(j_u \mid s_u) - \lambda \mathcal{H}(\pi(\cdot \mid s_u)) + (V_\psi(s_u) - \log c(u))^2 \right], \quad (15)$$

with entropy weight  $\lambda = 0.01$ . Only  $\phi$  and  $\psi$  are trained; the trunk and the marginal head are fixed, so the marginal used for the first value and for the analysis of Section 3.4 is unchanged. The critic is pre-trained on decisions recorded under the supervised policy before the actor is updated, and its held-out rank correlation with the true cost is used as a gate: if the baseline cannot rank costs, the advantage is noise and fine-tuning is not attempted. Epoch selection uses the total subtree cost on held-out training-distribution states, never the evaluation instances. At deployment the policy is used greedily:  $j^* = \arg \max_j g_j(s)$ .

### 3.9. Cost model

The comparison that motivates the design is per-node cost for a whole-instance verdict, and its structure is asymptotic rather than a constant factor. Let  $n_c$  be the number of free variables at a node. The network solves *one* relaxation to build its features and performs *one* forward pass, so its LP count is  $O(1)$  in  $n_c$ . LP-probing solves one relaxation per candidate variable,  $O(n_c)$ . The ratio between them therefore grows linearly in instance size, and any single number quoted for it is meaningful only together with the  $n_c$  at which it was measured.

This also means the comparison is sensitive to how the LP backend is implemented, in a way that cuts against the learned method. A backend that rebuilds the model for every call inflates probing’s cost by its full  $O(n_c)$  factor while charging the network only once, exaggerating the advantage. A

397 correctly engineered branch-and-bound keeps one relaxation and re-optimizes  
 398 from the parent basis after a bound change, which is an order of magnitude  
 399 cheaper per probe. Section 4.5 measures both methods against a warm-  
 400 started backend for this reason, and reports the ratio as a function of  $n_c$   
 401 rather than as a single figure.

## 402 4. Experiments

### 403 4.1. Instances and control of solution multiplicity

404 Instances are generated by the procedure GENHARD( $m, n, K$ ): draw a  
 405 random 0/1 matrix  $A$  of density 1/2; plant  $K = 20$  candidate solutions  
 406  $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(K)}$  with  $\mathbf{b}^{(k)} = A\mathbf{x}^{(k)}$ ; retain the one maximizing *vertex spread*,  
 407 the mean pairwise  $L_2$  distance among relaxation vertices obtained from ran-  
 408 dom objective directions. Maximizing vertex spread enlarges  $\tilde{\mathcal{S}}$  directly and  
 409 is the mechanism that makes market-split instances hard for relaxation-based  
 410 methods (Section 2). Uniqueness of the planted solution is not enforced. Ev-  
 411 ery instance is feasible by construction:  $\mathcal{S} \neq \emptyset$  is guaranteed by the planting,  
 412 and deciding infeasibility is outside the scope of this paper (Section 4.14).

413 *Why  $|\mathcal{S}|$  must be controlled across sizes..* Solution multiplicity changes the  
 414 *character* of the prediction task, not only its difficulty. When  $|\mathcal{S}| = 1$  every  
 415 free variable is DET and predicting well is tantamount to solving; when  $|\mathcal{S}|$  is  
 416 large most variables are OPEN and the task is to estimate a diffuse posterior.  
 417 Comparing sizes without controlling  $|\mathcal{S}|$  therefore confounds size with task  
 418 identity. Since  $|\mathcal{S}|$  falls as  $m$  rises at fixed  $n$ , we select  $m$  per size to match  
 419 it.

| Size           | $m$ | $m/n$ | median $ \mathcal{S} $ | root ceiling | conditional ceiling |
|----------------|-----|-------|------------------------|--------------|---------------------|
| $10 \times 25$ | 10  | 0.40  | 23                     | 70.4%        | 87.0%               |
| $18 \times 50$ | 18  | 0.36  | 19                     | 71.3%        | 87.2%               |
| $21 \times 60$ | 21  | 0.35  | 10                     | —            | —                   |

Table 1: Sizes used, with solution multiplicity matched. Ceilings (the Bayes-optimal per-variable accuracies of Section 4.3) agree to within one point between the two training sizes, so size is the variable that changes.

420 *Limits of the control..* Matching  $|\mathcal{S}|$  forces  $m/n$  to vary ( $0.40 \rightarrow 0.35$ ); at  
 421 fixed  $n$  the two cannot be held simultaneously and we prioritize  $|\mathcal{S}|$  because  
 422 it determines task identity. The control is also granular: at  $n = 60$  the  
 423 median  $|\mathcal{S}|$  steps  $42 \rightarrow 12 \rightarrow 2$  across  $m = 20, 21, 22$ , so  $m = 21$  is the best  
 424 available integer and the realized evaluation set has median  $|\mathcal{S}| = 10$  with  
 425 27/30 verified.

426 *Where enumeration stops..* At  $n = 70$  no enumeration completed within 120s  
 427 at the  $m$  giving the target multiplicity (0/10). At  $n = 100$  the obstruction  
 428 is more basic: a constraint solver finds *zero* solutions within 20s at  $m \in$   
 429  $\{28, 32, 36, 40\}$  despite a planted one. At that size the bottleneck is finding  
 430 any solution rather than choosing branches well, so a branching comparison  
 431 there would not be informative. We therefore evaluate at  $n \leq 60$  and report  
 432 this boundary as a limitation.

#### 433 4.2. Implementation and protocol

434 *Software..* Relaxations used for training features and for the original search  
 435 loop are solved with SCIP via PySCIPOpt; the warm-started relaxations  
 436 of Sections 4.5 and 4.6 use Gurobi 13 (dual simplex, bounds changed in  
 437 place), which requires a licence — HiGHS exposes an equivalent incremental  
 438 interface and should serve as a free substitute, though we have not verified  
 439 the timings with it. Enumeration and the CP-SAT baseline use OR-Tools;  
 440 lattice reduction uses fpylll (LLL and BKZ). The network is implemented in  
 441 PyTorch with PyTorch Geometric (GATConv). One property of the relaxation  
 442 deserves note: it carries no objective, so its solution is an arbitrary vertex  
 443 of the relaxation polytope and different LP codes return different vertices.  
 444 Both branching arms consume the same vertex within an experiment, so  
 445 comparisons are fair, but absolute node counts differ between the SCIP- and  
 446 Gurobi-backed runs.

447 *Training..* Adam, learning rate  $10^{-3}$ , 20 epochs over 10,000 in-tree states,  
 448 batch size 1 (instances vary in size), gradient-norm clipping at 1.0, loss (8)  
 449 with exact marginal targets. Training states are drawn at depths 0–12 uni-  
 450 formly, 6 states per source instance, from a pool of 1,760 instances; evaluation  
 451 uses 440 instances from a disjoint pool. Training takes approximately 20 min-  
 452 utes on one CPU core. *No hyperparameter was tuned*: width, depth, learning  
 453 rate, epoch count and the training depth range are the values first tried, and  
 454 there is no validation split. Reported figures are therefore not contaminated

by selection on the test set, but neither is any of these settings claimed to be optimal.

*Threading..* All timing-sensitive components run single-threaded (`torch.set_num_threads(1)`). On graphs of this size multi-threaded BLAS is counter-productive: we measure 11.3ms per forward pass at 8 threads against 0.9ms at one.

*Search evaluation..* Each arm is run on identical instances with a hard per-instance wall-clock budget enforced by subprocess termination, since lattice reduction executes inside a blocking native call that in-process timers cannot interrupt. Instances run at concurrency 6 on a 20-core machine. This low concurrency is deliberate: wall-clock-budgeted comparisons are sensitive to CPU contention, and at higher concurrency we have observed instances solvable in 14–15s standalone exceeding a 20s cap.

*Statistics..* Solve-rate differences between arms on the same instances are tested with McNemar’s exact test; per-instance speed differences with a two-sided sign test. We report both rather than a single aggregate because they can have different power. The main comparison at  $21 \times 60$  is replicated over three independent seeds and we report the spread; smaller-size and transfer results are single-run and marked as such. Seeds are set explicitly: an earlier version derived them from `hash()` of the instance filename, which Python randomizes per process, so those runs had no recorded provenance.

*Evaluation sets..* Two  $21 \times 60$  sets are used: the original 30 instances (generator seeds 700000–700029) and a 100-instance set (seeds 700000–700099) that contains them, built after the 30-instance results had been reported;  $|\mathcal{S}|$  is verified for 92/100, median 11. All numbers on the 100-instance set are lattice-off, three seeds. The training pool for the experiments of Sections 4.10 and 4.11 is a separate set of 500 instances at  $21 \times 60$  (seeds 800000–800499, split 400/100) whose overlap with every evaluation set is zero by hash of  $(A, \mathbf{b})$ ; the 100 held-out instances of that pool supply the states on which fine-tuning epochs are selected.

#### 4.3. Prediction quality against the Bayes ceiling

Because  $\mathcal{S}$  is enumerated exactly, the Bayes-optimal per-variable accuracy is computable:

$$\text{ceil} = \frac{1}{|K|} \sum_{j \in K} \max(p_j, 1 - p_j), \quad (16)$$

attained by answering the majority value of the surviving solutions. No deterministic predictor of  $\mathbf{x}^*$  from  $(A, \mathbf{b})$  can exceed it. Table 2 evaluates predictors against it on 400 held-out  $10 \times 25$  instances with  $\mathcal{S}$  fully enumerated.

| Predictor              | all variables | top-3        | $L_1$ to true marginal |
|------------------------|---------------|--------------|------------------------|
| Bayes ceiling          | <b>70.8%</b>  | <b>93.2%</b> | 0.0000                 |
| Proposed network       | 66.7%         | 86.6%        | 0.1216                 |
| LP relaxation rounding | 60.9%         | 65.8%        | —                      |

Table 2: Root-level prediction against the exact posterior on the 182 root states of the held-out  $10 \times 25$  evaluation pool. The network is the same checkpoint used in every search experiment below. “top-3” restricts to the three variables the predictor is most confident about.

The network comes within 4.1 points of the ceiling overall and 6.6 on the top-3 subset, and it recovers most of the gap between relaxation rounding and the ceiling ( $60.9 \rightarrow 66.7$  against a ceiling of 70.8). Two preliminary experiments on an earlier pool of  $10 \times 25$  instances, reported for completeness but not from the checkpoint evaluated here, bear on design choices: a 2.11M-parameter network with four output heads (selection, assignment, value and feasibility) scored 60.7% on root states, statistically indistinguishable from relaxation rounding at 60.6%, so capacity is not the binding constraint and a single-purpose head is preferable; and training the single-head architecture on exact marginals rather than the planted solution improved  $L_1$  to the posterior from 0.1261 to 0.0977 while leaving argmax accuracy unchanged, which matters because the analysis of Section 3.4 consumes the marginal rather than its argmax.

*What the aggregate conceals..* Table 3 decomposes the same root variables by the partition of Section 3.4.

At the root, 93.7% of variables are OPEN. The aggregate ceiling of 69.9% is therefore dominated by predictions that cannot affect the search, and on the 6.3% that can, the information is already complete (100%). Root-level accuracy is consequently a weak objective for a branching heuristic, and the operative question is not whether the information exists but what detects it — taken up next.

| Root variables                                 | count | share | ceiling on subset |
|------------------------------------------------|-------|-------|-------------------|
| DET (branching against it empties the subtree) | 38    | 6.3%  | 100.0%            |
| OPEN (either branch reaches a solution)        | 562   | 93.7% | 67.2%             |
| all variables                                  | 600   | 100%  | 69.3%             |

Table 3: Root-level decomposition, 24 instances of  $10 \times 25$  with  $\mathcal{S}$  enumerated. The aggregate is 94% composed of variables on which no branching decision can cost a backtrack.

#### 513 4.4. Depth, and where the residual lies

514 Using Proposition 1 we measure, at each depth along a prefix drawn  
515 from an actual solution, the conditional ceiling and the fraction of remaining  
516 variables that propagation forces (Table 4).

| depth      | surviving $ \mathcal{S} $ | conditional ceiling | proposed network | LP rounding |
|------------|---------------------------|---------------------|------------------|-------------|
| 0          | 25.3                      | 70.8%               | 66.7%            | 60.9%       |
| 4          | 4.2                       | 82.3%               | 72.8%            | 68.5%       |
| 7          | 1.8                       | 91.5%               | 81.2%            | 77.8%       |
| 8          | 1.4                       | 94.4%               | 84.3%            | 80.9%       |
| 12         | 1.1                       | 98.8%               | 96.8%            | 95.5%       |
| all depths | —                         | 87.2%               | 80.0%            | 76.5%       |

Table 4: Conditional information against depth on the held-out in-tree states of the  $10 \times 25$  evaluation pool ( $n = 25$ ), LP-confidence variable ordering. Same checkpoint as Table 2.

517 The consequential movement is in composition rather than level: the  
518 DET share rises from 6.3% at the root to 87.9% at depth 8 (Table 6). At the  
519 root almost every branching decision is inconsequential because both children  
520 contain solutions; by depth 8 almost every decision empties a subtree when  
521 wrong. The network’s margin over relaxation rounding is 3–6 points through-  
522 out and does not vanish at depth, which is the first indication that what it  
523 captures is not simply depth-dependent; Section 4.9 makes this precise.

524 *Does in-tree supervision matter, controlling for everything else?* A compari-  
525 son between a root-trained and an in-tree-trained network is only informative  
526 if nothing else differs. Table 5 holds the instance pool, the label type (exact  
527 marginals), the architecture, the optimizer and the number of training states

528 (1,760 each) fixed, and varies only whether the states are root states or in-  
529 tree states drawn at depths 0–12. Both are evaluated on the same held-out  
530 in-tree states.

| depth      | conditional ceiling | root-trained | in-tree-trained | LP rounding |
|------------|---------------------|--------------|-----------------|-------------|
| 0          | 70.7%               | 67.8%        | 66.5%           | 60.9%       |
| 4          | 82.1%               | 71.4%        | 72.4%           | 68.3%       |
| 6          | 89.7%               | 74.6%        | 78.2%           | 74.0%       |
| 8          | 94.4%               | 77.4%        | 84.2%           | 81.3%       |
| 10         | 97.1%               | 81.7%        | 91.4%           | 89.4%       |
| 12         | 98.8%               | 85.8%        | 96.5%           | 95.3%       |
| all depths | 87.2%               | 75.4%        | <b>79.8%</b>    | 76.4%       |

Table 5: Controlled comparison of training-state distribution. Same pool, labels, architecture, optimizer and state count; only the depth at which states are sampled differs. Evaluated on held-out in-tree states of the  $10 \times 25$  pool.

531 In-tree supervision is worth 4.4 points overall. The more telling row is the  
532 root-trained network itself: at 75.4% it is *below* relaxation rounding on in-  
533 tree states, so a network trained on root states does not merely underperform  
534 in the tree, it is worse than the classical heuristic it is meant to replace. The  
535 shift from root to interior states is not a distribution the root-trained network  
536 generalizes across, and in-tree training is what makes the learned guidance  
537 better than the baseline at all.

538 Table 6 decomposes the residual gap at these depths and asks what each  
539 method detects.

| depth | % DET | network on DET | propagation | LP integrality | LP-probing | network on OPEN |
|-------|-------|----------------|-------------|----------------|------------|-----------------|
| 5     | 61.9% | 89.7%          | 1.2%        | 67.9%          | 51.6%      | 54.4%           |
| 6     | 74.2% | 88.2%          | 1.6%        | 70.4%          | 68.0%      | 52.5%           |
| 7     | 79.9% | 88.8%          | 3.7%        | 73.3%          | 72.8%      | 51.2%           |
| 8     | 87.9% | 89.4%          | 5.3%        | 76.9%          | 79.8%      | 53.8%           |

Table 6: Residual decomposition on the same states as Table 4. Columns 4–6 are recall on the ground-truth DET set (LP-probing measured with the warm-started backend of Section 4.5); columns 3 and 7 are the network’s argmax accuracy on each subset.

540 The gap lies entirely on DET variables: at depth 7,  $0.80 \times 11.2 \approx 9$  points,  
541 matching the measured 10.3-point shortfall against the conditional ceiling.

542 Performance near 50% on OPEN variables is correct behaviour rather than  
543 a defect, since those variables are genuinely undetermined. The striking  
544 figure is that unit propagation detects only 1.2–5.3% of logically determined  
545 variables: the information is present and derivable but not reachable by the  
546 cheapest sound rule. LP-probing recovers 52–80% and LP integrality 68–  
547 77%.

#### 548 4.5. Cost per node

549 Both methods are charged everything they must pay to reach a whole-  
550 instance verdict: the network one relaxation for its features plus one forward  
551 pass, probing one relaxation per free variable. Both use the same warm-  
552 started backend, a single model whose variable bounds are changed in place  
553 and re-optimized from the parent basis (Gurobi dual simplex), which is how a  
554 branch-and-bound implementation reuses its relaxation. Timings are single-  
555 threaded.

| Instances                                                               | $n$ | network | LP-probing | ratio                          | probing recall |
|-------------------------------------------------------------------------|-----|---------|------------|--------------------------------|----------------|
| $10 \times 25$                                                          | 25  | 3.22 ms | 5.60 ms    | $1.7\times$                    | —              |
| $18 \times 50$                                                          | 50  | 2.33 ms | 11.84 ms   | $5.1\times$                    | —              |
| $21 \times 60$                                                          | 60  | 2.57 ms | 18.39 ms   | $7.2\times$                    | —              |
| $40 \times 100$                                                         | 100 | 4.02 ms | 62.46 ms   | $15.5\times$                   | —              |
| $60 \times 150$                                                         | 150 | 4.45 ms | 79.06 ms   | <b><math>17.8\times</math></b> | —              |
| <i>in-tree states of <math>10 \times 25</math> instances, by depth:</i> |     |         |            |                                |                |
| depth 5                                                                 | 20  | 2.44 ms | 4.02 ms    | $1.6\times$                    | 51.6%          |
| depth 6                                                                 | 19  | 2.39 ms | 3.98 ms    | $1.7\times$                    | 68.0%          |
| depth 7                                                                 | 18  | 2.35 ms | 3.43 ms    | $1.5\times$                    | 72.8%          |
| depth 8                                                                 | 17  | 2.34 ms | 3.13 ms    | $1.3\times$                    | 79.8%          |

Table 7: Per-node cost for a whole-instance verdict, warm-started LP backend for both methods, single-threaded. The network’s accuracy on DET variables at depths 5–8 is 85.9–92.7% against probing’s recall in the last column.

556 The ratio behaves as the cost model predicts: it is negligible at  $n = 25$   
557 and reaches  $17.8\times$  at  $n = 150$ , tracking the  $O(n_c)$  versus  $O(1)$  difference  
558 in relaxation count. Two consequences follow, and they point in opposite  
559 directions.

560 *The advantage is real but asymptotic, and small where we can search..* At the  
 561 sizes where our search actually operates ( $n = 25\text{--}60$ ) the ratio is  $1.3\text{--}7.2\times$ .  
 562 At the depths where prediction matters most (5–8 of a  $10 \times 25$  instance,  
 563  $n_c = 17\text{--}20$ ) it is  $1.3\text{--}1.7\times$ . A practitioner choosing between the two at  
 564 that scale should prefer probing, which is *sound*: where its test fires the  
 565 conclusion is a proof, whereas the network never offers a guarantee. The  
 566 learned alternative becomes compelling only at  $n \gtrsim 100$ , and Section 4.1  
 567 shows that search itself fails there for every strategy we tried.

568 *The measurement is sensitive to baseline engineering..* An earlier version of  
 569 this comparison used a backend that rebuilt the LP model on every call  
 570 and, additionally, excluded the network’s own feature relaxation from its  
 571 timer. That combination reported  $19\text{--}23\times$  at depths 5–8, where the correctly  
 572 measured value is  $1.3\text{--}1.7\times$  — an overstatement of more than an order of  
 573 magnitude, arising entirely from how the two sides were instrumented rather  
 574 than from what they compute. We report the corrected figures and note  
 575 the general hazard: in any cost comparison against a classical procedure, the  
 576 classical side must be implemented as a practitioner would implement it, and  
 577 the learned side must be charged for its own preprocessing.

#### 578 4.6. Search performance

579 We compare branching strategies under matched wall-clock budgets with  
 580 the lattice stage disabled, isolating branching quality. LP selects the variable  
 581 with the most integral relaxation value and tries that value first; MODEL uses  
 582 the proposed network. Both run in the same loop: one warm-started relax-  
 583 ation per node (Section 4.2), identical propagation, identical value ordering,  
 584 so the arms differ only in which variable they branch on and which value  
 585 they try first. Each configuration is run three times with different seeds; the  
 586 depth-first search itself is deterministic given the relaxation vertex, so seeds  
 587 change timing but not node counts.

588 Three points. First, the learned guidance produces a smaller tree at every  
 589 size —  $2.0\times$ ,  $3.9\times$  and  $9.5\times$  fewer nodes than LP — and the ratio grows with  
 590 size, as the analysis of Section 4.9 predicts: the states where the network  
 591 separates from relaxation rounding are the large, weakly constrained ones  
 592 near the top of a big tree, and  $10 \times 25$  has few of them. Second, a smaller  
 593 tree buys time only once the tree is large enough to pay for the forward pass.  
 594 At  $10 \times 25$  the MODEL arm is *slower* ( $0.34\text{--}0.43\times$ , faster on at most  $1/30$   
 595 instances), because a 24-node tree costs less than the network evaluations

| Size (budget)          | strategy | solve rate      | median nodes | median time                         | nodes/s |
|------------------------|----------|-----------------|--------------|-------------------------------------|---------|
| $10 \times 25$ (60 s)  | LP       | $100 \pm 0.0\%$ | 46           | <b><math>0.03 \pm 0.00</math> s</b> | 1,767   |
|                        | MODEL    | $100 \pm 0.0\%$ | <b>24</b>    | $0.07 \pm 0.01$ s                   | 376     |
| $18 \times 50$ (300 s) | LP       | $100 \pm 0.0\%$ | 4,224        | $2.43 \pm 0.33$ s                   | 2,066   |
|                        | MODEL    | $100 \pm 0.0\%$ | <b>1,085</b> | <b><math>1.82 \pm 0.37</math> s</b> | 731     |
| $21 \times 60$ (600 s) | LP       | $100 \pm 0.0\%$ | 82,366       | $51.5 \pm 2.0$ s                    | 1,627   |
|                        | MODEL    | $100 \pm 0.0\%$ | <b>8,657</b> | <b><math>14.6 \pm 0.7</math> s</b>  | 619     |

Table 8: Branching strategy comparison on the warm-started loop, lattice stage disabled, 30 instances per cell, mean  $\pm$  standard deviation over three seeds of the per-seed median. Node counts are seed-invariant.

that shrink it; its per-node cost is  $2.6\text{--}4.7\times$  that of LP at every size, which is the forward pass of Section 4.5. At  $18 \times 50$  the median per-instance ratio is  $1.30\text{--}1.37\times$  in the network’s favour but not significant (faster on 18/30 in every seed, sign test  $p = 0.36$ ). Third, at  $21 \times 60$  the advantage is clear and replicates: the median per-instance ratio is  $2.57\times$ ,  $3.03\times$  and  $2.96\times$  across seeds, the network is faster on exactly 23/30 instances in each ( $p = 0.005$ ), and both arms solve every instance within budget.

Two further strategies were measured on an earlier version of this loop that rebuilt the relaxation at every node. RANDOM branching solved 0/30 at  $20 \times 50$  within 300 s, and Section 4.12 shows it closes none of the  $21 \times 60$  residual instances on the present loop either. LP-PROBE, which probes every free variable and soundly fixes the forced ones before branching as LP, produced by far the smallest trees ( $51\times$  fewer nodes than LP at  $20 \times 50$ ) and was the slowest arm, processing 6 nodes/s against 376: node count and wall-clock rank the methods in opposite orders, and a tree-size convention would select the worst of them. We have not repeated LP-PROBE on the warm-started loop; Section 4.5 measures its per-node cost there directly.

*Replication on 100 instances..* Thirty instances leave the sign test with little power and the median with wide uncertainty, so the  $21 \times 60$  comparison was repeated on the 100-instance set (Table 9). The direction and significance hold — MODEL is faster on 70–71/100 instances ( $p < 0.001$ ) with  $5.5\times$  fewer nodes — but the margins are smaller than on the original thirty:  $2.0\text{--}2.3\times$  in time against  $2.6\text{--}3.0\times$ , and  $5.5\times$  in nodes against  $9.5\times$ . The original thirty are the first thirty seeds of the larger set, and on them the supervised

620 policy happens to do unusually well; we treat the 100-instance figures as  
 621 the estimate and the 30-instance ones as the favourable end of its sampling  
 622 variation.

| strategy                | solve rate | median nodes | median time        | nodes/s | vs. LP (per instance)            |
|-------------------------|------------|--------------|--------------------|---------|----------------------------------|
| LP                      | 100 ± 0.0% | 79,609       | 41.2 ± 1.9 s       | 1,966   | —                                |
| MODEL                   | 100 ± 0.0% | 13,386       | 16.5 ± 0.6 s       | 796     | 2.0–2.3×, 70–71/100, $p < 0.001$ |
| MODEL+FT (Section 4.11) | 100 ± 0.0% | <b>5,495</b> | <b>7.6 ± 0.3 s</b> | 738     | 4.8–5.9×, 81–84/100, $p < 0.001$ |

Table 9:  $21 \times 60$ , 100 instances, lattice off, 600 s budget, three seeds (mean ± s.d. of per-seed medians; node counts are seed-invariant). The last row is the cost-aware fine-tuned policy of Section 4.11.

623 We also record what the rebuilt-relaxation loop reported for the  $21 \times 60$   
 624 comparison, because an earlier draft of this paper rested on it: 30/30 against  
 625 25/30 solved, and a 3.29× median speedup. Warm-starting the relaxation  
 626 removed the solve-rate difference entirely — the five instances LP failed within  
 627 600 s it now closes in 65–270 s — and reduced the speedup. The solve-rate  
 628 advantage was an artifact of a slow backend that penalized the arm expanding  
 629 more nodes, and we do not claim it.

#### 630 4.7. Comparison against complete solvers

631 Table 8 compares branching strategies inside our own search loop, which  
 632 measures branching quality but says nothing about how the system stands  
 633 against an off-the-shelf solver. That comparison is reported here, on the same  
 634 instances, single-threaded, under the same budget.

| Size               | CP-SAT         | SCIP    | proposed (MODEL)            | CP-SAT advantage |
|--------------------|----------------|---------|-----------------------------|------------------|
| 10 × 25            | <b>0.002 s</b> | 0.009 s | 0.07 s                      | 35×              |
| 18 × 50            | <b>0.046 s</b> | 0.385 s | 1.82 s                      | 40×              |
| 21 × 60            | <b>1.159 s</b> | 3.687 s | 14.6 s                      | 13×              |
| 21 × 60, 100 inst. | <b>0.792 s</b> | 2.590 s | 16.5 s / 7.6 s (fine-tuned) | 21× / 10×        |

Table 10: Median solve time; all methods solve every instance at every size. The proposed column is the MODEL arm of Table 8 (mean of per-seed medians); the last row adds the 100-instance set and the fine-tuned policy of Section 4.11. The proposed system does not beat a complete solver at any size tested, and is outperformed by SCIP, whose LP code it uses for training features.

635 The gap is large and we state it plainly: *the contribution of this paper is*  
 636 *not a solver that competes with CP-SAT*. It is worth asking where the gap

comes from, because the answer bears on what the branching heuristic is actually doing. Table 11 decomposes it at  $21 \times 60$  into tree size and per-node throughput, using CP-SAT’s own branch counter.

|                     | CP-SAT                | proposed     | ratio                    |
|---------------------|-----------------------|--------------|--------------------------|
| nodes (median)      | 18,827 branches       | <b>8,657</b> | ours $2.2\times$ fewer   |
| throughput          | <b>19,782 nodes/s</b> | 619 nodes/s  | CP-SAT $32\times$ faster |
| total time (median) | 1.065 s               | 14.6 s       | $14\times$               |

Table 11: Decomposition of the  $21 \times 60$  gap. CP-SAT additionally learns a median of 5,514 conflict clauses per instance.

The learned guidance produces a *smaller* search tree than CP-SAT —  $2.2\times$  fewer nodes despite CP-SAT’s clause learning — and loses entirely on per-node cost. Where that cost goes can be measured. Before the loop was warm-started it processed 254 nodes/s, spending 4.34 ms per node rebuilding and solving the relaxation; warm-starting the same LP brings that to 0.08 ms and raises throughput to 619 nodes/s, which is the loop reported throughout Section 4.6. Of the 1.6 ms that remain per node, the network’s forward pass is 1.88 ms at the root and less at deeper nodes where the reduced graph is smaller, and propagation and bookkeeping are under 0.1 ms: the forward pass now accounts for over 90% of per-node cost, and we have not reduced it. We therefore do not claim that the gap is an implementation artifact. What the node counts establish is narrower: the deficit is not in branching quality. Closing it would require at minimum a compiled inference path and, as the next paragraph notes, algorithmic components our search lacks.

Two honest caveats attach to this reading. Tree sizes are not perfectly commensurable — CP-SAT’s “branches” include restarts and conflict-driven jumps, and each of its nodes performs more propagation work than ours — so the  $2.2\times$  should be read as an order-of-magnitude statement. And a competitive implementation would need more than a faster forward pass: clause learning and restarts are absent from our search, and the last two are what let CP-SAT scale past the sizes where we stop.

#### 4.8. Size transfer

Because the features are size-normalized and the parameters do not depend on  $(m, n)$ , one network applies at any size. We test whether that transfers in practice, with  $|\mathcal{S}|$  matched as in Table 1.

| Training size  | Evaluation size | solve rate | median time       | median nodes |
|----------------|-----------------|------------|-------------------|--------------|
| $10 \times 25$ | $10 \times 25$  | 100%       | $0.07 \pm 0.01$ s | 24           |
| $10 \times 25$ | $18 \times 50$  | 100%       | $1.82 \pm 0.37$ s | 1,085        |
| $18 \times 50$ | $18 \times 50$  | 100%       | $1.48 \pm 0.15$ s | 942          |
| $10 \times 25$ | $21 \times 60$  | 100%       | $14.6 \pm 0.7$ s  | 8,657        |
| $18 \times 50$ | $21 \times 60$  | 100%       | $23.7 \pm 0.7$ s  | 14,742       |

Table 12: Transfer on the warm-started loop, lattice stage disabled, 30 instances per row, three seeds. Rows 2–3 and 4–5 each hold the evaluation size fixed and vary only the training size.

At the  $21 \times 60$  evaluation size the network trained only at  $10 \times 25$  — a  $2.4 \times$  smaller problem — is not worse than the one trained at  $18 \times 50$ ; if anything it is better, expanding fewer nodes on  $18/30$  instances and finishing faster on  $18\text{--}19/30$  in each seed, with a median per-instance time ratio of  $1.24\text{--}1.31 \times$  in its favour. Neither difference is significant (sign test  $p \geq 0.20$ ), and both solve  $30/30$ . At  $18 \times 50$  the picture is the same: the  $10 \times 25$  network is faster on  $16\text{--}17/30$  instances ( $p \geq 0.59$ ) although its median time is slightly higher, the two summaries disagreeing because the paired differences are small against instance-to-instance variation. A previous version of this comparison, on the rebuilt-relaxation loop, gave  $15/30$  and a  $0.92 \times$  ratio; the direction is not stable across loops and we read the result as no detectable transfer loss rather than as a transfer gain. Root-level prediction agrees: the  $10 \times 25$  network scores 67.8%, 69.5% and 69.4% at  $10 \times 25$ ,  $20 \times 50$  and  $40 \times 100$ , holding its margin over relaxation rounding (+11.0, +7.2, +4.7 points) at every size.

*Transfer of both stages beyond  $21 \times 60$ .* Table 13 adds sizes above the training range of every learned component, with  $m$  chosen for the constraint density of  $21 \times 60$  ( $m/n \approx 0.35$ ) because solution multiplicity cannot be verified by enumeration there. Stage 1 keeps a node advantage over the LP rule at  $24 \times 70$  ( $3.8 \times$ ) and  $28 \times 80$  ( $2.4 \times$ ) but its time advantage vanishes at  $24 \times 70$  ( $0.7\text{--}1.1 \times$  on instances both solve), since a 260,000-node tree pays for as many forward passes. The cost-fine-tuned policies (Section 4.11), trained at  $21 \times 60$  and at  $18 \times 50$  only, solve  $30/30$  at  $24 \times 70$  in every seed,  $2.3\text{--}3.3 \times$  and  $2.1\text{--}2.7 \times$  faster than Stage 1, and at  $28 \times 80$  solve 18 and 17 of 30 within 1,200 s against 8 for Stage 1 and 4 for the LP rule, where SCIP solves all 30 in a median of 112 s; that size is the upper edge of the useful window.

| size                     | measure              | CP-SAT       | SCIP         | LP                   | MODEL (25)        | MODEL+FT (60)            | MODEL+FT (50)     |
|--------------------------|----------------------|--------------|--------------|----------------------|-------------------|--------------------------|-------------------|
| 24 × 70 (1200s, 3 seeds) | solved / median time | 30/30 / 16s  | 30/30 / 12s  | 23–25/30 / 537–644 s | 29/30 / 358–457 s | <b>30/30 / 125–185 s</b> | 30/30 / 135–165 s |
|                          | median nodes         | —            | —            | 980,743              | 260,114           | <b>90,054</b>            | 95,818            |
| 28 × 80 (1200s, 1 seed)  | solved / median time | 23/30 / 270s | 30/30 / 112s | 4/30 / >1200s        | 8/30 / >1200s     | <b>18/30 / 720s</b>      | 17/30 / 852s      |
|                          | median nodes         | —            | —            | 1,929,041            | 803,535           | <b>470,606</b>           | 560,077           |

Table 13: Sizes above every training range, lattice off, 30 instances each; solution multiplicity unverified, constraint density matched. Training sizes in parentheses.

691 This matters practically because label generation is the binding con-  
692 straint. Exact marginals require enumeration, which fails beyond  $n \approx 60$ ;  
693 transfer means a network trained where labels are obtainable can be deployed  
694 where they are not.

#### 695 4.9. Where the transferred network helps

696 Section 4.4 located the useful prediction regime at depths 6–8 of a  $10 \times 25$   
697 instance. Whether that localization survives transfer is a separate question,  
698 and it does not: at  $21 \times 60$  the search queries the network on states with 5–  
699 60 free variables, only 19.7% of which fall inside the training range of 13–25,  
700 and 80.2% of which are larger than any training state. Table 14 evaluates  
701 the network and relaxation rounding against exact conditional marginals on  
702  $21 \times 60$  in-tree states, grouped by the number of free variables, on the subset  
703 of states that could be enumerated within 15 s.

| free vars | in training range | $m/ K $ | network on DET | LP on DET | gain        | corr(network, LP) |
|-----------|-------------------|---------|----------------|-----------|-------------|-------------------|
| 20        | yes               | 1.05    | 100.0%         | 100.0%    | +0.0        | 1.000             |
| 25        | yes               | 0.84    | 100.0%         | 100.0%    | +0.0        | 0.998             |
| 30        | no                | 0.70    | 95.7%          | 95.0%     | +0.7        | 0.954             |
| 40        | no                | 0.53    | 75.5%          | 75.2%     | +0.4        | 0.747             |
| 50        | no                | 0.42    | 70.2%          | 65.0%     | <b>+5.3</b> | 0.664             |
| 60        | no                | 0.35    | 93.3%          | 88.9%     | <b>+4.4</b> | 0.658             |

Table 14:  $21 \times 60$  in-tree states by number of free variables, 9–14 enumerable states per row. Gain is the network’s margin over relaxation rounding on DET variables; the last column is the Pearson correlation between the two predictors’ outputs.

704 The pattern is the reverse of what a depth-based account predicts. Inside  
705 the training range the network is indistinguishable from relaxation round-  
706 ing — both are perfect and their outputs correlate at 1.000 — because at  
707  $m/|K| \geq 0.8$  the reduced instance is over-constrained and the relaxation is  
708 already integral on every DET variable. The network’s entire margin arises  
709 on states it never saw during training, at 50–60 free variables, where the

710 relaxation is weakest ( $m/|K| \approx 0.35\text{--}0.42$ ) and its outputs have decorrelated  
711 from the relaxation’s. In terms of Section 3.4, this is the region where DET  
712 variables are present but the relaxation does not find them.

713 Two conclusions follow. The first corrects Section 4.4: depth was a proxy  
714 for constraint density that happened to hold on  $10 \times 25$ , where descending  
715 from the root moves  $m/|K|$  only from 0.40 to 0.59, and does not hold on  
716  $21 \times 60$ , where the same descent moves it from 0.35 past 1.0. The useful  
717 regime is defined by  $m/|K|$ , and at deployment size it sits near the *top* of the  
718 tree. The second is that the transfer result of Section 4.8 is stronger than a  
719 matched-multiplicity comparison alone shows: the network extrapolates to  
720 states outside its training range in both size and constraint density, and that  
721 extrapolation is where its contribution lives. It is also a limitation, since the  
722 training range was not chosen with this in mind (Section 4.14).

#### 723 4.10. Training on deployment-sized states: a negative result

724 Section 4.9 located the network’s margin on states with 50–60 free vari-  
725 ables, outside the 13–25 range of its training states, and Section 4.14 of an  
726 earlier draft listed aligning the two as an obvious untested improvement. We  
727 tested it, holding everything but the training-state distribution fixed.

728 *States..* From a separate  $21 \times 60$  pool (Section 4.2) we built in-tree states with  
729 30, 35, . . . , 60 free variables by fixing a prefix of the LP-confidence ordering  
730 to the planted solution, and labelled them with exact conditional marginals  
731 under the enumeration rule of Section 3.5. Every state with at most 55  
732 free variables enumerated within 90 s; at 60 (the root) 291/400 did, and the  
733 failures are the instances with large  $|\mathcal{S}|$ , so root labels are biased toward  
734 small solution sets. A property of these states matters for what follows: with  
735 the prefix fixed to a solution, the surviving solution set is almost always a  
736 singleton once 50 or fewer variables remain (median  $|\mathcal{S}(F, \mathbf{v})| = 1$ , DET share  
737 97%), so their labels are nearly hard. Three networks were trained with the  
738 architecture, optimizer, epochs and loss of Section 4.2: M0, the network used  
739 everywhere above ( $10 \times 25$  in-tree states); M1, the 2,691 deployment-range  
740 states only; M2, half of each. A pre-diagnostic on  $10 \times 25$  alone, training  
741 on depth ranges 0–6, 6–12 and 0–12 with equal state counts, gave in-tree  
742 accuracies of 79.3%, 79.9% and 79.9%: within one size, the depth range does  
743 not matter.

744 *Prediction..* Re-running the analysis of Table 14 with the three networks  
745 scored on identical states, the gain over relaxation rounding on DET variables

at 50 and 60 free variables is +5.4 and +9.7 points for M0, +6.1 and +10.7 for M1, +5.9 and +10.6 for M2. Training in the deployment regime buys about one point there.

*Search..* It costs far more elsewhere. On the 100-instance set M1 expands a median of 21,337 nodes against 13,386 for M0 ( $1.6\times$  more; fewer on only 38/100,  $p = 0.021$ ) and is  $1.7\text{--}1.8\times$  slower; on the original thirty it expands 20,746 against 8,657. M2 sits between (18,558,  $p = 0.089$ ). A switch that routes nodes with  $\geq 50$  free variables to M1 and the rest to M0 recovers nothing (17,252 nodes, 43/99,  $p = 0.23$ ). The mechanism is visible in the prediction diagnostics: on small, dense interior states ( $10 \times 25$  at depth 6–8, the kind the search spends two thirds of its nodes on) the probability that M1’s most confident variable is DET and predicted correctly falls from 94.4% to 77.6%, and the rank correlation between its confidence and the true margin from 0.36 to 0.12, while on the deployment-range states it improves ( $77.2 \rightarrow 86.0\%$ ). Nodes are where the interior is, and a network that is slightly better near the top of the tree and much worse inside it makes the search slower. We conclude that the training-state size distribution is not the lever, and that the residual identified in Section 4.9 is not a distribution-shift artifact.

#### 4.11. Cost-aware fine-tuning

If neither accuracy on DET variables nor the training distribution explains what the supervised guidance leaves on the table, the natural suspect is the objective: the rule branches on the variable it is surest about, and being sure is not the same as being cheap. Three measurements test this, and the third turns it into a method.

*R0: does the confident choice cost the least?.* On 100 held-out states with 45–50 free variables we forced the root decision to each of the five variables the network is most confident about, ran the supervised search below it, and recorded the exact subtree cost of each. The confidence rule’s choice is the cheapest of the five on 22% of states; the median ratio of its cost to the cheapest candidate is 1.29 and the mean 2.97; on 24% of states some candidate is at least twice as cheap; and choosing the cheapest candidate at the root alone would remove 55% of all nodes summed over these states. The LP rule’s root choice costs the same as the network’s on only 10% of states, so the two rules are not merely tied. There is a large residual, and it is about *which* variable to branch on.

782 *R1: can the cost be predicted?*. The critic is the baseline of the policy update,  
783 so it must rank decisions by cost on states it has not seen. Recording every  
784 decision of the supervised search on the training pool gave 37,275 decisions  
785 (9,375 on held-out states) with their exact subtree costs, of which two thirds  
786 occur at 25–34 free variables. A value head on the frozen trunk reaches a  
787 held-out Spearman correlation of 0.80 with  $\log c$ ; a baseline that knows only  
788 the state’s size (a linear fit on  $\log |K|$ ) reaches 0.70, and within size buckets  
789 the critic’s advantage is larger (0.68 against 0.38 at 35–44 free variables,  
790 0.57 against 0.37 at 45–55). Unfreezing the trunk adds 0.02. The gate of  
791 Section 3.8 is passed with the frozen critic.

792 *R2: fine-tuning.*. Six epochs of 400 rollouts each ( $\tau = 1$ , 90 s budget, about  
793 21,000 decisions per epoch) from the 45–60 free-variable training states. On  
794 the 150 held-out monitor states the total subtree cost under the greedy policy,  
795 relative to the supervised rule, was 0.79, 0.49, 0.56, 0.46, 0.65 and 0.56 after  
796 epochs 1–6; epoch 4 was selected. Policy entropy fell from 1.73 to 0.10 over  
797 the same epochs, so the policy is near-deterministic by the selected epoch.

798 *Evaluation.*. Table 9 reports the fine-tuned policy (MODEL+FT) beside the  
799 supervised one on the 100-instance set: 5,495 nodes against 13,386 ( $2.3\times$   
800 fewer, fewer on 70/100,  $p < 0.001$  in every seed) and a median of 7.6 s against  
801 16.5 s ( $2.0\text{--}2.2\times$ , faster on 64–67/100,  $p \leq 0.007$ ), at an unchanged per-node  
802 cost (738 against 796 nodes/s). Against LP-guided branching this is  $4.8\text{--}5.9\times$   
803 in time and  $14\times$  in nodes. On the original thirty instances the effect is small  
804 (8,873 against 8,657 nodes;  $1.2\text{--}1.5\times$  in time, 18–21/30,  $p$  between 0.04 and  
805 0.36): the seventy added instances account for the gain ( $2.5\times$  fewer nodes,  
806 53/70), which is consistent with the thirty being a favourable sample for the  
807 supervised rule. In the deployed cascade (Table 15) the fine-tuned policy  
808 closes the lattice residual in 7.3–11.7 s against 12.2–17.7 s for the supervised  
809 one.

810 *Replication and transfer of the fine-tuned policy.*. A second fine-tuning run  
811 with a different training seed (different rollout sampling and initialization  
812 of the correction head; epoch 2 selected by the same monitor) gives the  
813 same picture on the 100-instance set — 4,940 nodes ( $2.4\times$  fewer than the  
814 supervised policy, fewer on 68/100,  $p < 0.001$ ) and  $1.9\text{--}2.4\times$  faster (66–  
815 68/100,  $p \leq 0.002$ ) — and a larger one on the original thirty (4,040 nodes,  
816  $1.9\times$  fewer on 19/30;  $2.3\times$  faster, 20–21/30). The two runs select different  
817 variables at many nodes and reach similar costs, which is what one expects

818 if the objective, not a particular solution, is what was learned. The policy  
819 was fine-tuned on  $21 \times 60$  states only, yet at  $18 \times 50$  it expands 424 nodes  
820 against the supervised policy’s 1,085 ( $1.7\times$  fewer, 20/30,  $p = 0.099$ ) and is  
821  $1.8\text{--}2.2\times$  faster (22–24/30,  $p \leq 0.016$ ): the cost-aware improvement transfers  
822 downward in size as the supervised one does.

823 *What changed..* On 300 held-out states the fine-tuned policy branches on a  
824 different root variable than the confidence rule at 95% of them. Its pick is  
825 typically the 15th most confident variable ( $|\hat{p} - \frac{1}{2}|$  of 0.26 against 0.43 for the  
826 rule’s pick), and it is DET with a correct first value less often (57.5% against  
827 82.5%). The policy has moved *away* from the variables it is surest about,  
828 and the search got cheaper. The improvement is not attributable to the root  
829 decision alone: forcing only the root to the fine-tuned pick and running the  
830 supervised policy below it gives a median cost ratio of 0.93 (cheaper on 22/46  
831 disagreement states), so the gain is a property of the sequence of decisions  
832 rather than of any single one. We can therefore say what the fine-tuning  
833 optimizes — subtree cost, with the probability of being right held as a soft  
834 prior — but not yet name the feature it found; a cheap proxy, the amount of  
835 propagation the first child triggers, does not differ between the two choices.

#### 836 4.12. Deployment in the full pipeline

837 Table 8 isolates branching by disabling lattice reduction. In deployment  
838 the lattice stage runs first and the search sees only its residual. Because  
839 that stage is independent of the branching heuristic, every arm receives the  
840 identical residual set.

| Size           | strategy   | closed by lattice | residual solved by search | median time on residual | end-to-end       |
|----------------|------------|-------------------|---------------------------|-------------------------|------------------|
| $10 \times 25$ | both       | 30/30             | —                         | —                       | 100%             |
| $18 \times 50$ | LP / MODEL | 24–28/30          | 11/11 / 11/11             | 0.5–5.0 s / 0.6–2.5 s   | 100% / 100%      |
| $21 \times 60$ | RANDOM     | 10–16/30          | 0/50                      | > 600 s                 | $44.4 \pm 8.3\%$ |
| $21 \times 60$ | LP         | 10–16/30          | 50/50                     | 52.9–65.1 s             | $100 \pm 0.0\%$  |
| $21 \times 60$ | MODEL      | 10–16/30          | 50/50                     | 14.2–20.2 s             | $100 \pm 0.0\%$  |
| $21 \times 60$ | MODEL+FT   | 10–16/30          | 50/50                     | <b>7.3–11.7 s</b>       | $100 \pm 0.0\%$  |

Table 15: Cascade results on the warm-started loop, three seeds at every size, 600 s budget at  $21 \times 60$ . Lattice coverage varies across seeds (16, 14, 10 of 30 at  $21 \times 60$ ) because the lattice stage draws random column permutations, so the residual handed to the search varies with it; all arms within a seed receive the identical residual.

841 Lattice coverage falls with size — 30/30, 24–28/30, 10–16/30 — and that  
842 decline is what creates room for branching quality to matter. At  $10 \times 25$

and  $18 \times 50$  the residual is empty or closed within seconds by either sound strategy. At  $21 \times 60$  between ten and twenty instances reach the search depending on the seed, and the residual is genuinely hard: random branching closes none of the 50 residual instances pooled over seeds, so the end-to-end rate of that arm is the lattice coverage alone. Both relaxation-based and learned branching close all 50, and the difference between them is time. On the residual the learned guidance is faster on 11/14, 14/16 and 15/20 instances in the three seeds (median per-instance ratio  $3.48\times$ ,  $2.99\times$ ,  $2.38\times$ ; sign test  $p = 0.057$ ,  $0.004$ ,  $0.041$ ), 40/50 pooled with a median ratio of  $3.00\times$  ( $p = 2.4 \times 10^{-5}$ ), expanding 6.9–9.5 $\times$  fewer nodes. The residual is where the branching heuristic operates in deployment, and the speedup there matches the isolated measurement of Table 8.

#### 4.13. Does the graph structure contribute?

MarginalNet consumes six scalars per node and propagates them over the bipartite graph. With a feature set that small, a per-variable model ignoring the graph is a serious baseline: if it matches, the message passing is decoration. Table 16 compares models trained on identical targets, loss and splits, differing only in whether structure is used. The aggregated variant appends mean and max of the constraint-side features over each variable’s incident rows, which is the information one message-passing round would deliver.

| Model                          | accuracy     | $L_1$ to marginal | parameters |
|--------------------------------|--------------|-------------------|------------|
| Linear, variable features only | 78.1%        | 0.2530            | <b>4</b>   |
| MLP, variable features only    | 77.8%        | 0.2522            | 8,641      |
| MLP + constraint aggregates    | 79.4%        | 0.2361            | 9,025      |
| MarginalNet (bipartite graph)  | <b>84.1%</b> | <b>0.1514</b>     | 21,761     |
| Bayes ceiling                  | 87.3%        | 0.0000            | —          |

Table 16: Ablation on in-tree states of  $10 \times 25$  instances, 2,000 training and 800 test states.

Message passing contributes: +4.7 points over the best structure-free model and a 36% reduction in  $L_1$ , closing roughly 60% of the residual gap to the ceiling that the MLP leaves open. The improvement in  $L_1$  is the more relevant of the two, since the analysis of Section 3.4 consumes the marginal rather than its argmax.

869 The same table bounds the claim, though. A four-parameter linear model  
870 on the relaxation value alone reaches 78.1%, within 9.2 points of the ceiling;  
871 most of the available signal is in that single feature, and the graph adds a  
872 useful but not dominant increment on top of it.

#### 873 4.14. Limitations

874 *Label generation bounds the method, not only its evaluation..* Exact marginals  
875 require enumerating  $\mathcal{S}$ , which does not complete beyond  $n \approx 60$  on this fam-  
876 ily. Transfer permits deployment at larger sizes but not training there; ap-  
877 proximating  $p_j$  by solution sampling is the natural extension and is untested.

878 *The useful size window is bounded on both sides..* Below it, lattice reduction  
879 closes everything and branching quality is irrelevant. Above it the search  
880 fails: at  $28 \times 80$  the fine-tuned policies solve 18/30 within 1,200s and the su-  
881 pervised rule 8/30, and at  $n = 100$  a constraint solver finds no solution within  
882 20s. Our demonstration lives at  $n = 50$ –70; sizes above 60 are matched on  
883 constraint density only and  $28 \times 80$  is a single seed.

884 *The advantage is in time and tree size, not in solve rate..* On a warm-started  
885 loop both sound strategies solve every instance we evaluate within budget;  
886 what the supervised guidance buys at  $21 \times 60$  is a 2.0–2.3 $\times$  median speedup  
887 and 5.5 $\times$  fewer nodes on 100 instances (2.6–3.0 $\times$  and 9.5 $\times$  on the original  
888 thirty, a favourable sample), and the fine-tuned policy a further 2.0–2.2 $\times$   
889 and 2.3 $\times$ , significant per instance in every seed. An earlier draft reported  
890 a 5–0 solve-rate advantage from a loop that rebuilt the relaxation at every  
891 node; that difference vanished with the faster backend and we do not claim it.  
892 Whether a solve-rate advantage exists at larger budgets or sizes is untested,  
893 and with 30 evaluation instances per size the power to detect one would be  
894 low in any case.

895 *Multiplicity control is approximate..* Median  $|\mathcal{S}|$  is 23, 19 and 10 across the  
896 three sizes; at  $n = 60$  multiplicity steps  $42 \rightarrow 12 \rightarrow 2$  across consecutive inte-  
897 ger  $m$ , so finer matching is unavailable. Three of thirty evaluation instances  
898 have unverified  $|\mathcal{S}|$  and are marked as such. Matching  $|\mathcal{S}|$  also moves  $m/n$   
899 from 0.40 to 0.35.

900 *Cost advantage depends on scale and on baseline engineering..* The amortiza-  
901 tion argument holds asymptotically but is worth little at the sizes where our  
902 search operates: 1.3–1.7 $\times$  at the depths where prediction matters, against a

903 warm-started probing baseline that is additionally sound. It reaches 15–18×  
 904 only at  $n \geq 100$ , where search fails for every strategy we tested. We therefore  
 905 cannot demonstrate the regime in which the argument is strongest.

906 *The search implementation is not competitive..* At  $21 \times 60$  CP-SAT is 9–10×  
 907 faster than the fine-tuned policy and 20× faster than the supervised one on  
 908 the 100-instance set (13× on the original thirty). Our branching produces  
 909 fewer nodes, so the deficit is per-node throughput (32×), now dominated by  
 910 the network’s forward pass; we have not shown that a competitive implemen-  
 911 tation is achievable: a compiled inference path, clause learning and restarts  
 912 are all absent, and the latter two are load-bearing for CP-SAT at larger sizes.

913 *Training and deployment distributions barely overlap, and aligning them did*  
 914 *not help..* The training range  $\{0, \dots, 12\}$  produces states with 13–25 free  
 915 variables, whereas the  $21 \times 60$  search queries states with 5–60 and spends  
 916 80% of its queries above 25. Section 4.10 shows that training on the deploy-  
 917 ment regime makes the search slower; we do not know whether a different  
 918 construction of large training states (prefixes from the search’s own trajec-  
 919 tory rather than from a solution, or states with several surviving solutions)  
 920 would behave differently.

921 *The fine-tuning result rests on one training pool and few training seeds..* The  
 922 policy was fine-tuned on states from one  $21 \times 60$  pool; two training seeds agree  
 923 at  $21 \times 60$  and the gain transfers to  $18 \times 50$ , but no larger size was tested  
 924 and both seeds select their epoch on the same monitor. Its entropy collapses  
 925 within four epochs, epoch selection is by a held-out monitor rather than  
 926 by any principled stopping rule, and we cannot yet name the feature the  
 927 policy exploits (Section 4.11). Whether the gain transfers across sizes as the  
 928 supervised one does is untested.

929 *Hyperparameters are untuned and there is no validation split for the super-*  
 930 *vised network..* See Section 4.2. The configuration is defensible as a first  
 931 attempt and is not contaminated by the test set; the depth range, at least,  
 932 was subsequently shown not to matter within a size (Section 4.10). The fine-  
 933 tuning stage does select an epoch on held-out states, which are drawn from  
 934 the training pool and disjoint from every evaluation instance.

935 *Every instance is feasible; deciding infeasibility is out of scope..* The condi-  
 936 tional marginal is undefined on an empty solution set, so Stage 1’s supervision

937 requires  $\mathcal{S} \neq \emptyset$ , and every instance we train or evaluate on has a planted so-  
 938 lution. Finding a solution quickly and refuting an infeasible instance quickly  
 939 are different objectives, and the second is where clause learning dominates.  
 940 Stage 2’s signal, by contrast, is defined either way — the cost of a decision  
 941 is the size of its subtree whether that subtree contains a solution or is ex-  
 942 hausted — so extending the method to infeasible instances is naturally a  
 943 Stage-2 question, and it is untested.

944 *Single instance family.* All results concern one generator from one appli-  
 945 cation. Whether the amortization argument transfers to other constraint  
 946 families is untested.

## 947 5. Conclusion

948 We presented a learned branching heuristic for binary linear systems to-  
 949 gether with an exact procedure for supervising it. Two elements carry the  
 950 method. Conditioning a BLS on a partial assignment is identical to reducing  
 951 it, so a single network trained on ordinary instances predicts at every node of  
 952 the search tree without architectural change. And on instances small enough  
 953 to enumerate, the exact conditional posterior both supplies the training tar-  
 954 get and identifies which predictions can affect the search at all: only variables  
 955 that every surviving solution agrees on can cost a backtrack, and their share  
 956 rises from 6.3% at the root to 87.9% at depth 8.

957 Measured inside our search loop the heuristic does what it is designed  
 958 to do. It is  $2.0\text{--}2.3\times$  faster in median than relaxation-based branching at  
 959  $21 \times 60$  on 100 instances in each of three seeds, expands  $5.5\times$  fewer nodes,  
 960 and closes the residual of the deployed pipeline  $3.0\times$  faster, where random  
 961 branching closes nothing; with solution multiplicity controlled, a network  
 962 trained at  $2.4\times$  smaller scale performs indistinguishably from one trained at  
 963 the evaluation size, which matters because exact supervision is available only  
 964 at small sizes.

965 We are equally explicit about what the results do not establish. The sys-  
 966 tem does not compete with a complete solver: CP-SAT is  $9\text{--}40\times$  faster at  
 967 every size we tested. Decomposing that gap is informative rather than excul-  
 968 patory — our branching yields  $2.2\times$  fewer nodes and loses on throughput by  
 969  $32\times$ , which places the deficit in a Python research implementation whose per-  
 970 node cost is now the network’s forward pass, and which lacks clause learning  
 971 and restarts — but we have not shown that closing it is achievable. Nor does

972 the cost argument carry the weight we initially assigned it. Against a prop-  
 973 erly warm-started probing baseline the advantage is  $1.3\text{--}1.7\times$  at the depths  
 974 where prediction matters on our instances, and probing is sound where the  
 975 network is not; the ratio reaches  $15\text{--}18\times$  only at  $n \geq 100$ , a regime in which  
 976 search fails for every strategy we tried. An earlier version of this compar-  
 977 ison, using a backend that rebuilt the LP for each probe and excluded the  
 978 network’s own feature solve from its timer, reported  $19\text{--}23\times$  at a scale where  
 979 the true figure is near unity, and we record the correction because the error  
 980 came entirely from instrumentation rather than from what the two methods  
 981 compute.

982 Two further results sharpen what the supervised target is. Moving the  
 983 training states to the deployment regime, where the network’s prediction  
 984 margin lives, does not help and in fact doubles the search tree, because the  
 985 search spends its nodes in the small interior states the moved network no  
 986 longer handles well; the training-state distribution is not the lever. Optimiz-  
 987 ing the cost the search pays is. With the exact subtree cost of every decision  
 988 read off the search, an actor-critic fine-tuning of the branching choice —  
 989 initialized at the supervised rule, with the marginal network frozen — ex-  
 990 pands  $2.3\times$  fewer nodes and runs  $2.0\text{--}2.2\times$  faster than the supervised policy  
 991 on 100 instances, and the policy it finds branches on variables the network  
 992 is *less* sure about. The probability that a branch is right and the cost of  
 993 the branch being wrong are different objectives, and the second is the one a  
 994 search minimizes.

995 What remains, and what we claim, is methodological. For planted-  
 996 solution instance families the exact conditional posterior is computable and is  
 997 a well-posed supervision signal for branching, where the conventional target  
 998 — a single planted solution — is not. States drawn from inside the search  
 999 tree are a better training distribution than root states, and the reduction  
 1000 identity makes generating them a matter of substitution rather than solver  
 1001 instrumentation. The resulting heuristic transfers across instance size when  
 1002 solution multiplicity is held comparable, and the exact cost of its decisions,  
 1003 available from the search, is a second well-posed training signal that im-  
 1004 proves it. Whether this becomes practically useful depends on two things  
 1005 we have not done: extending exact supervision past  $n \approx 60$ , most plausibly  
 1006 by approximating the marginals without exhaustive enumeration, and imple-  
 1007 menting the search well enough that a branching improvement is visible at  
 1008 the system level.

## 1009 Reproducibility

1010 Code is organized by component under `proposed_src/`, with shared mod-  
1011 ules in `util/` and a `ROLES.txt` in each directory describing every file. Scripts  
1012 are run from their own directory. Every search number in Tables 8, 12  
1013 and 15 comes from one chain, `v23_ablation/run_revision2.sh`, which runs  
1014 the warm-started loop (`v23_warm_search.py`) over all conditions and seeds;  
1015 `v23_aggregate.py` produces the tables from its output.

```
1016 # instances with matched |S| and exact marginals
1017 cd proposed_src/v15_bayes
1018 python3 v15_gen_by_m.py --m 10 --n 25 --count 2200 --K 20 \
1019     --out ../../runs/v22/sols_10x25.json
1020
1021 # attach solution sets, then split 80/20 (seed 0)
1022 cd ../v22_transfer
1023 python3 v22_add_solutions.py \
1024     --inp ../../runs/v22/sols_10x25.json \
1025     --out ../../runs/v22/full_10x25.json
1026 python3 v22_split.py # -> full_10x25_{train,test}
1027
1028 # in-tree training states (depths 0-12, 6 per instance)
1029 cd ../v17_conditional
1030 python3 v17_make_conditional_data.py \
1031     --sols ../../runs/v22/full_10x25_train.json \
1032     --max_depth 12 --per_instance 6 \
1033     --out ../../runs/v22/cond_10x25_train.json
1034
1035 # train (about 20 minutes, single CPU core)
1036 python3 v17_train_conditional.py \
1037     --train ../../runs/v22/cond_10x25_train.json \
1038     --test  ../../runs/v22/cond_10x25_test.json \
1039     --n_train 10000 --n_test 2000 --epochs 20 \
1040     --target soft --out ../../runs/model_n25
1041
1042 # every search table: warm-started loop, 3 seeds,
1043 # conditions A/B/R/C/D/Crand x {lattice off, on}
1044 cd ../v23_ablation
1045 bash run_revision2.sh # -> runs/v23/, chain.log
1046 python3 v23_aggregate.py # -> runs/v23/aggregate.json
```

```

1047
1048 # v24: range training (negative), 100-instance
1049 # set, R0/R1/R2 fine-tuning and evaluation
1050 cd ../v24_deploy_range
1051 bash run_v24.sh; bash run_v24_switch.sh
1052 bash run_v24_r1.sh; bash run_v24_r2.sh
1053 python3 v24_aggregate.py \
1054     --tags M0 M1 M2 SW PV

```

1055 All generators are seeded. The released checkpoints are 96 KB each. In-  
1056 stance data and weights are regenerated from the above rather than dis-  
1057 tributed, and the full evaluation cycle is scripted in `run_revision2.sh` under  
1058 `v23_ablation/`.

## 1059 References

- 1060 [1] Aardal, K., Hurkens, C. A. J., and Lenstra, A. K. (2000). Solving a  
1061 system of linear Diophantine equations with lower and upper bounds on  
1062 the variables. *Mathematics of Operations Research*, 25(3):427–442.
- 1063 [2] Aardal, K., Bixby, R. E., Hurkens, C. A. J., Lenstra, A. K., and  
1064 Smeltink, J. W. (2000). Market split and basis reduction: Towards a  
1065 solution of the Cornuéjols–Dawande instances. *INFORMS Journal on*  
1066 *Computing*, 12(3):192–202.
- 1067 [3] Amos, B. (2023). Tutorial on amortized optimization. *Foundations and*  
1068 *Trends in Machine Learning*, 16(5):592–732.
- 1069 [4] Bengio, Y., Lodi, A., and Prouvost, A. (2021). Machine learning for  
1070 combinatorial optimization: A methodological tour d’horizon. *European*  
1071 *Journal of Operational Research*, 290(2):405–421.
- 1072 [5] Chen, Z., Liu, J., Wang, X., Lu, J., and Yin, W. (2023). On represent-  
1073 ing mixed-integer linear programs by graph neural networks. In *Inter-*  
1074 *national Conference on Learning Representations*.
- 1075 [6] Cornuéjols, G. and Dawande, M. (1999). A class of hard small 0-1 pro-  
1076 grams. In *Integer Programming and Combinatorial Optimization*, pages  
1077 284–293.

- 1078 [7] Gasse, M., Chételat, D., Ferroni, N., Charlin, L., and Lodi, A. (2019).  
1079 Exact combinatorial optimization with graph convolutional neural net-  
1080 works. In *Advances in Neural Information Processing Systems*.
- 1081 [8] Gershman, S. J. and Goodman, N. D. (2014). Amortized inference in  
1082 probabilistic reasoning. In *Proceedings of the 36th Annual Meeting of  
1083 the Cognitive Science Society*, pages 517–522.
- 1084 [9] Khalil, E. B., Le Bodic, P., Song, L., Nemhauser, G., and Dilkina, B.  
1085 (2016). Learning to branch in mixed integer programming. In *AAAI  
1086 Conference on Artificial Intelligence*, pages 724–731.
- 1087 [10] Lenstra, A. K., Lenstra, H. W., and Lovász, L. (1982). Factoring poly-  
1088 nomials with rational coefficients. *Mathematische Annalen*, 261(4):515–  
1089 534.
- 1090 [11] Li, Z., Guo, J., and Si, X. (2023). G4SATBench: Benchmarking and  
1091 advancing SAT solving with graph neural networks. *Transactions on  
1092 Machine Learning Research*.
- 1093 [12] Marques-Silva, J. P. and Sakallah, K. A. (1999). GRASP: A search algo-  
1094 rithm for propositional satisfiability. *IEEE Transactions on Computers*,  
1095 48(5):506–521.
- 1096 [13] Millidge, B. (2022). Deconfusing direct vs amortized opti-  
1097 mization. Unrefereed technical note, [https://www.beren.io/  
1098 2022-09-25-Deconfusing-direct-vs-amortized-optimization/](https://www.beren.io/2022-09-25-Deconfusing-direct-vs-amortized-optimization/).
- 1099 [14] Moskewicz, M. W., Madigan, C. F., Zhao, Y., Zhang, L., and Malik, S.  
1100 (2001). Chaff: Engineering an efficient SAT solver. In *Design Automa-  
1101 tion Conference*, pages 530–535.
- 1102 [15] Ohrimenko, O., Stuckey, P. J., and Codish, M. (2009). Propagation via  
1103 lazy clause generation. *Constraints*, 14(3):357–391.
- 1104 [16] Schnorr, C. P. and Euchner, M. (1994). Lattice basis reduction: Im-  
1105 proved practical algorithms and solving subset sum problems. *Mathe-  
1106 matical Programming*, 66(1–3):181–199.
- 1107 [17] Selsam, D., Lamm, M., Bünz, B., Liang, P., de Moura, L., and Dill,  
1108 D. L. (2019). Learning a SAT solver from single-bit supervision. In *In-  
1109 ternational Conference on Learning Representations*.

- 1110 [18] Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Ben-  
1111 gio, Y. (2018). Graph attention networks. In *International Conference*  
1112 *on Learning Representations*.
- 1113 [19] Williams, R. J. (1992). Simple statistical gradient-following algorithms  
1114 for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–  
1115 256.
- 1116 [20] Etheve, M., Alès, Z., Bissuel, C., Juan, O., and Kedad-Sidhoum, S.  
1117 (2020). Reinforcement learning for variable selection in a branch and  
1118 bound algorithm. In *Integration of Constraint Programming, Artificial*  
1119 *Intelligence, and Operations Research (CPAIOR)*, pages 176–185.
- 1120 [21] Scavuzzo, L., Chen, F. Y., Chételat, D., Gasse, M., Lodi, A., Yorke-  
1121 Smith, N., and Aardal, K. (2022). Learning to branch with tree MDPs.  
1122 In *Advances in Neural Information Processing Systems*, volume 35.
- 1123 [22] Haralick, R. M. and Elliott, G. L. (1980). Increasing tree search efficiency  
1124 for constraint satisfaction problems. *Artificial Intelligence*, 14(3):263–  
1125 313.
- 1126 [23] Cappart, Q., Moisan, T., Rousseau, L.-M., Prémont-Schwarz, I., and  
1127 Cire, A. A. (2021). Combining reinforcement learning and constraint  
1128 programming for combinatorial optimization. In *AAAI Conference on*  
1129 *Artificial Intelligence*.
- 1130 [24] Chu, G. and Stuckey, P. J. (2015). Learning value heuristics for con-  
1131 straint programming. In *CPAIOR*.
- 1132 [25] Feng, S. and Yang, Y. (2025). SORREL: Suboptimal-demonstration-  
1133 guided reinforcement learning for learning to branch. In *AAAI Confer-*  
1134 *ence on Artificial Intelligence*.
- 1135 [26] Han, Q., Yang, L., Chen, Q., Zhou, X., Zhang, D., Wang, A., Sun,  
1136 R., and Luo, X. (2023). A GNN-guided predict-and-search framework  
1137 for mixed-integer linear programming. In *International Conference on*  
1138 *Learning Representations*.
- 1139 [27] Khalil, E. B., Morris, C., and Lodi, A. (2022). MIP-GNN: A data-driven  
1140 framework for guiding combinatorial solvers. In *AAAI Conference on*  
1141 *Artificial Intelligence*.

- 1142 [28] Li, Z. and Si, X. (2022). NSNet: A general neural probabilistic frame-  
1143 work for satisfiability problems. In *Advances in Neural Information Pro-*  
1144 *cessing Systems*, volume 35.
- 1145 [29] Montanari, A., Ricci-Tersenghi, F., and Semerjian, G. (2007). Solving  
1146 constraint satisfaction problems through belief propagation-guided dec-  
1147 imation. In *Allerton Conference on Communication, Control, and Com-*  
1148 *puting*.
- 1149 [30] Nair, V., Bartunov, S., Gimeno, F., et al. (2020). Solving mixed integer  
1150 programs using neural networks. *arXiv:2012.13349*.
- 1151 [31] Parsonson, C. W. F., Laterre, A., and Barrett, T. D. (2023). Reinforce-  
1152 ment learning for branch-and-bound optimisation using retrospective  
1153 trajectories. In *AAAI Conference on Artificial Intelligence*.
- 1154 [32] Pesant, G., Quimper, C.-G., and Zanarini, A. (2012). Counting-based  
1155 search: Branching heuristics for constraint satisfaction problems. *Jour-*  
1156 *nal of Artificial Intelligence Research*, 43:173–210.
- 1157 [33] Refalo, P. (2004). Impact-based search strategies for constraint pro-  
1158 gramming. In *Principles and Practice of Constraint Programming (CP)*,  
1159 pages 557–571.
- 1160 [34] Vaezipoor, P., Lederman, G., Wu, Y., Maddison, C., Grosse, R. B., Se-  
1161 shia, S. A., and Bacchus, F. (2021). Learning branching heuristics for  
1162 propositional model counting. In *AAAI Conference on Artificial Intelli-*  
1163 *gence*.
- 1164 [35] Wassermann, A. (2025). Solving the market split problem with lattice  
1165 enumeration. *arXiv:2508.08702*; to appear in *Mathematical Program-*  
1166 *ming Computation*.
- 1167 [36] Xu, K., Hu, W., Leskovec, J., and Jegelka, S. (2019). How powerful  
1168 are graph neural networks? In *International Conference on Learning*  
1169 *Representations*.